Java Developing Environment Setup
Introduction to Java Programming Language

Desenvolvimento de Software e Sistemas Móveis (DSSMV)
Licenciatura em Engenharia de Telecomunicações e Informática
LETI/ISEP

2026/27

Paulo Baltarejo Sousa and Carlos Filipe Freitas
{pbs,caf}@isep.ipp.pt



Disclaimer

Material and Slides

Some of the material/slides are adapted from various:

- Presentations found on the internet;
- Books;
- Web sites;
- ...

Outline

1. **IntelliJ IDEA**
   - Create a project
2. **Java Program Basics**
   - The Structure of a Simple Program
3. **Object Oriented Programming Basics**
   - Classes
   - Objects
4. **Fundamental Data Types**
   - Numerical
   - Operators
5. **Control Statements**
   - Decisions
   - Loops
   - Jump Statements
6. **Input and Output**
7. **Strings**
8. **Arrays**
9. **Array Lists**
10. **Converting to ...**
11. **Random Numbers**
12. **Programming Issues**
    - Primitive Data Types
    - Integer Division
13. **Unit Tests**
14. **Using a Debugger**

IntelliJ IDEA

Download

- Go to `https://www.jetbrains.com/idea/`
- Go to download page by clicking on **Download**.
- Choose the **Ultimate** version.
- Request the IntelliJ IDEA key in `https://www.jetbrains.com/shop/eform/students`
  - You have to use your **ISEP email address**.
- Follow the instructions to activate the IntelliJ IDEA.
  - The instructions are sent via email.

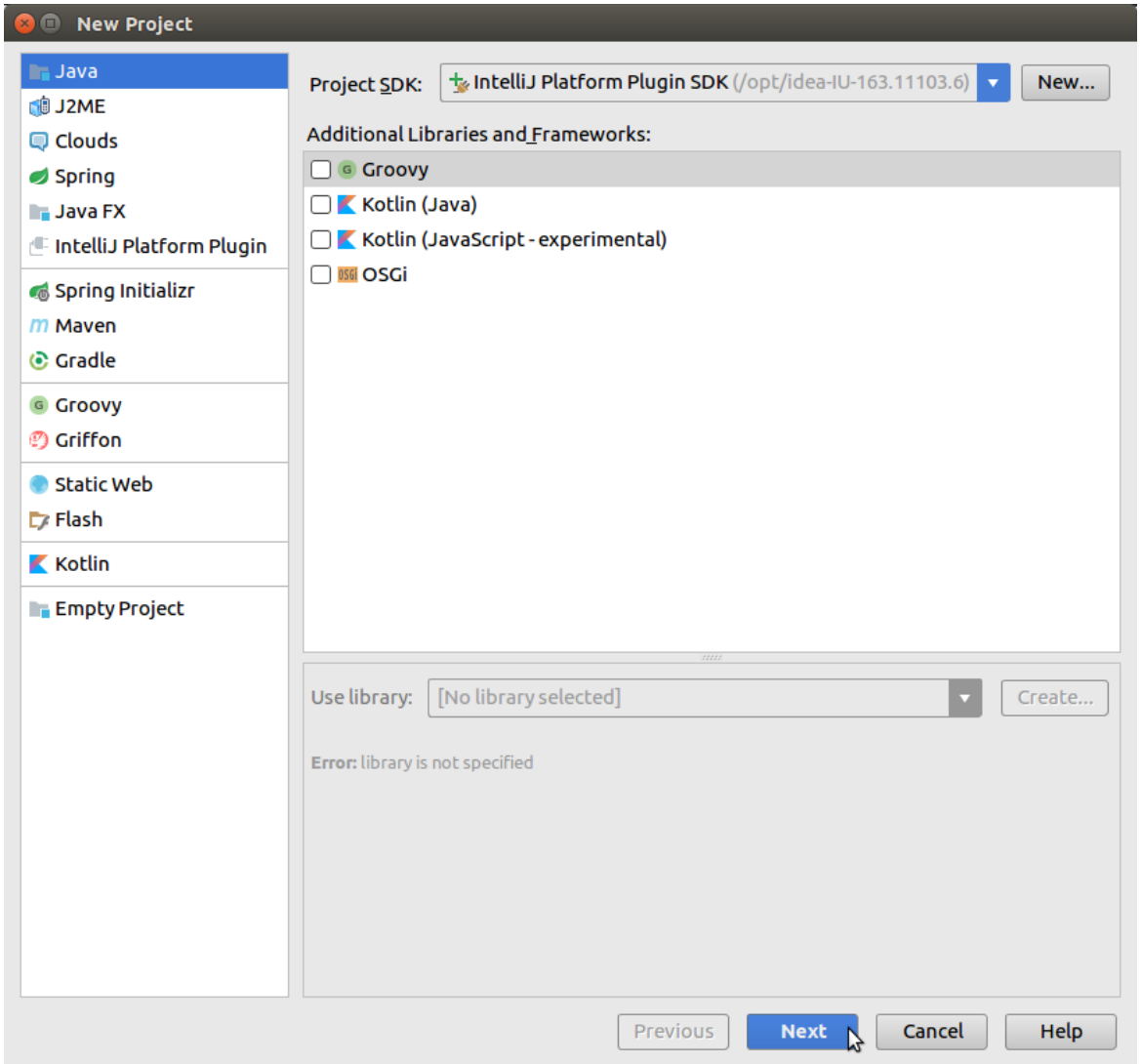
Installation

- Go to `https://www.jetbrains.com/idea/whatsnew/` and following the installation instructions.
  - Enable "Database Tools" and "Android".
- Discover IntelliJ IDEA in `https://www.jetbrains.com/help/idea/discover-intellij-idea.html` and `https://www.jetbrains.com/idea/documentation/`.

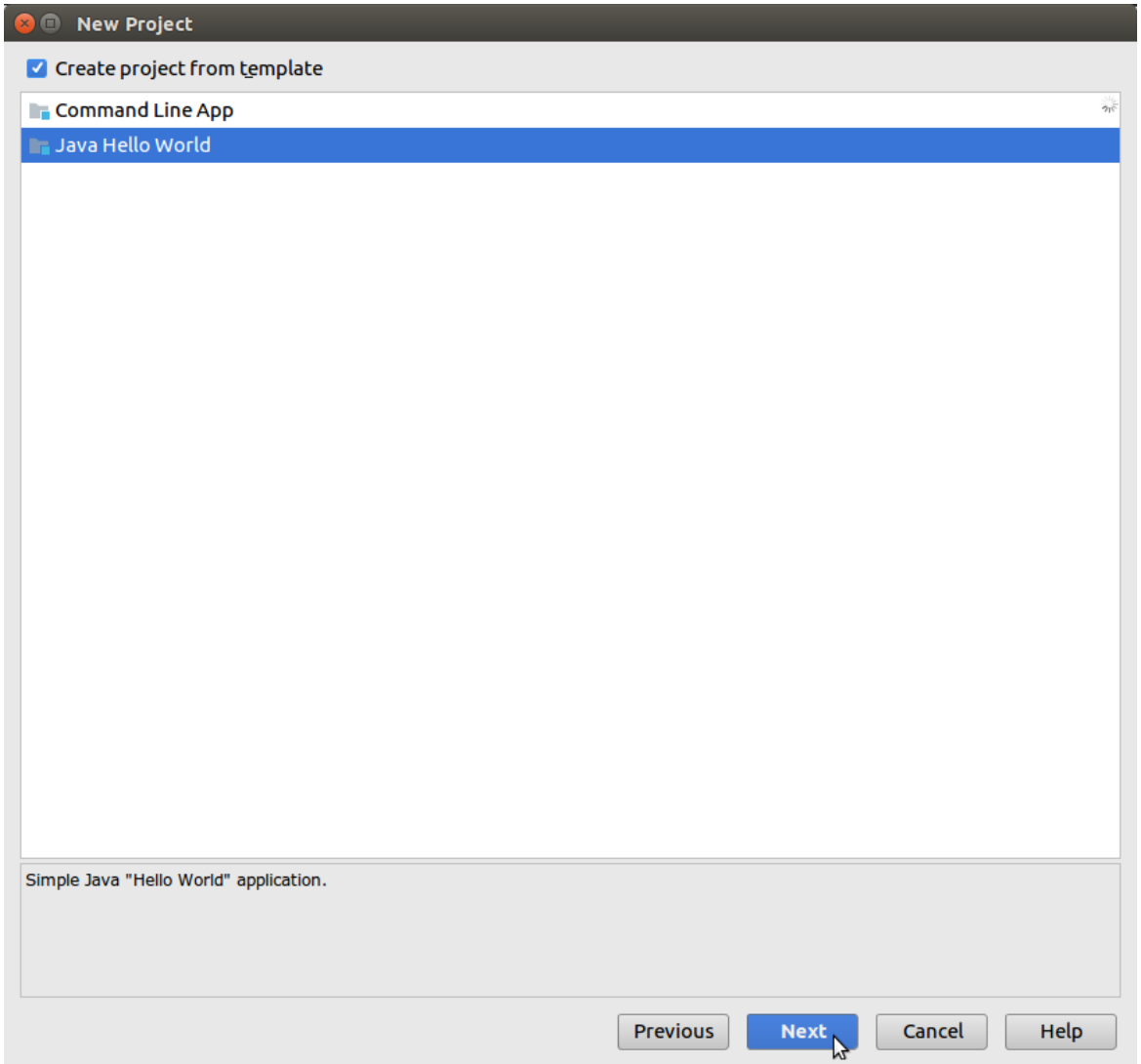
Welcome


Welcome to IntelliJ IDEA
IntelliJ IDEA
Version 2016.3.3
Create New Project
Import Project
Open
Check out from Version Control
Configure
Get Help

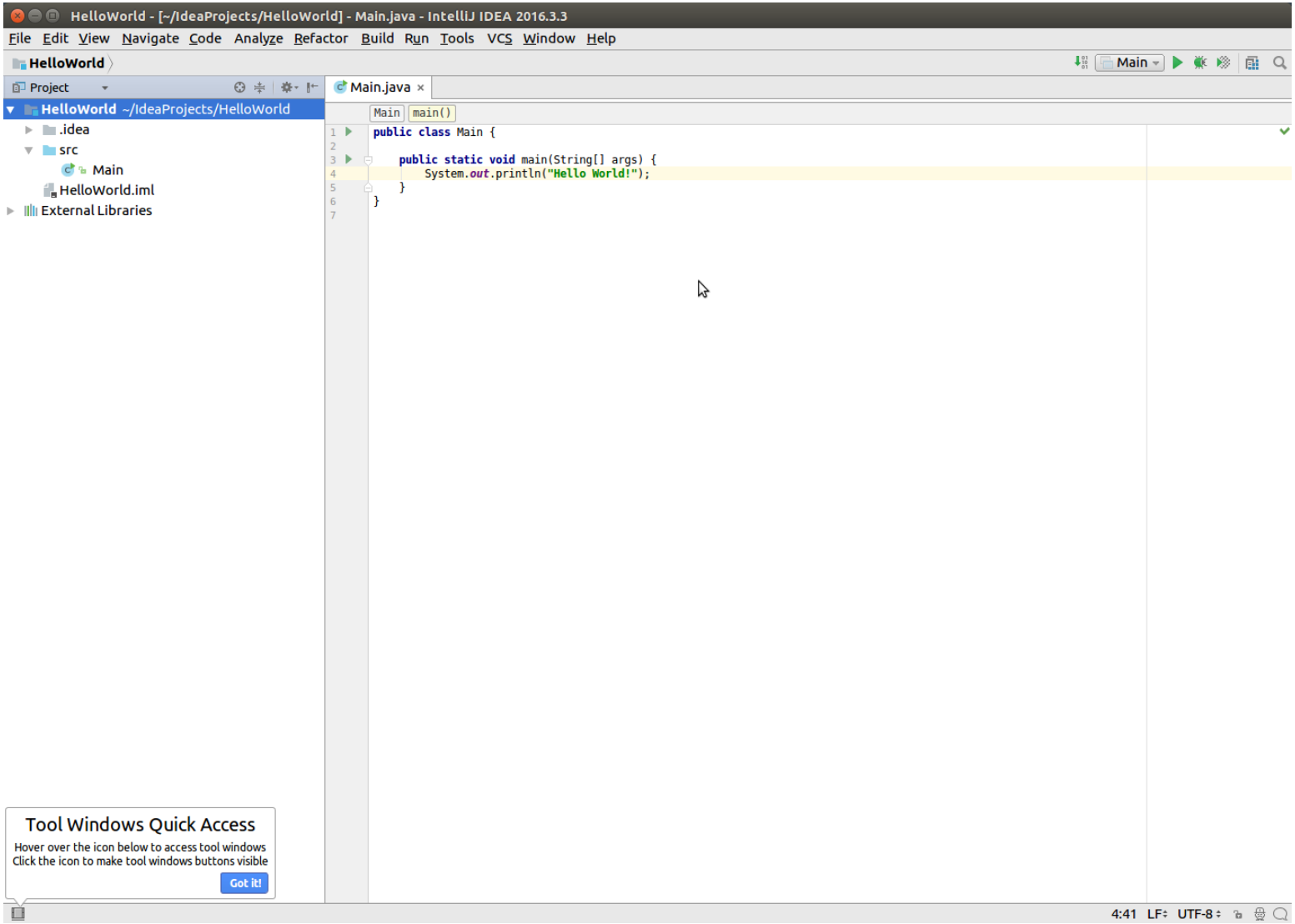
New Project


New Project
Java
J2ME
Clouds
Spring
Java FX
IntelliJ Platform Plugin
Spring Initializr
Maven
Gradle
Groovy
Griffon
Static Web
Flash
Kotlin
Empty Project
Project SDK:
IntelliJ Platform Plugin SDK (/opt/idea-IU-163.11103.6)
New...
Additional Libraries and Frameworks:
Groovy
Kotlin (Java)
Kotlin (JavaScript - experimental)
OSGi
Use library:
[No library selected]
Create...
Error: library is not specified
Previous
Next
Cancel
Help

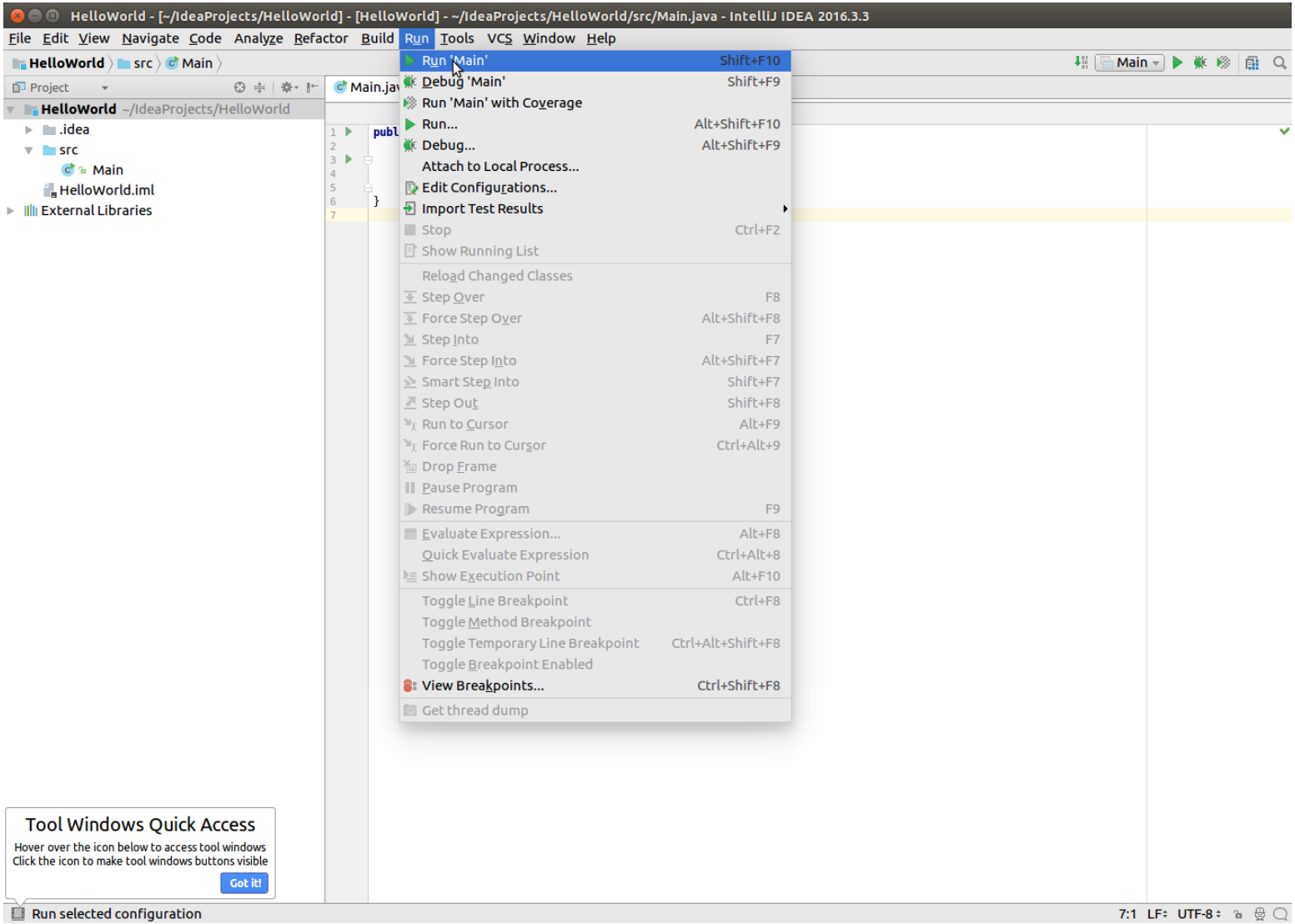
Choosing a template



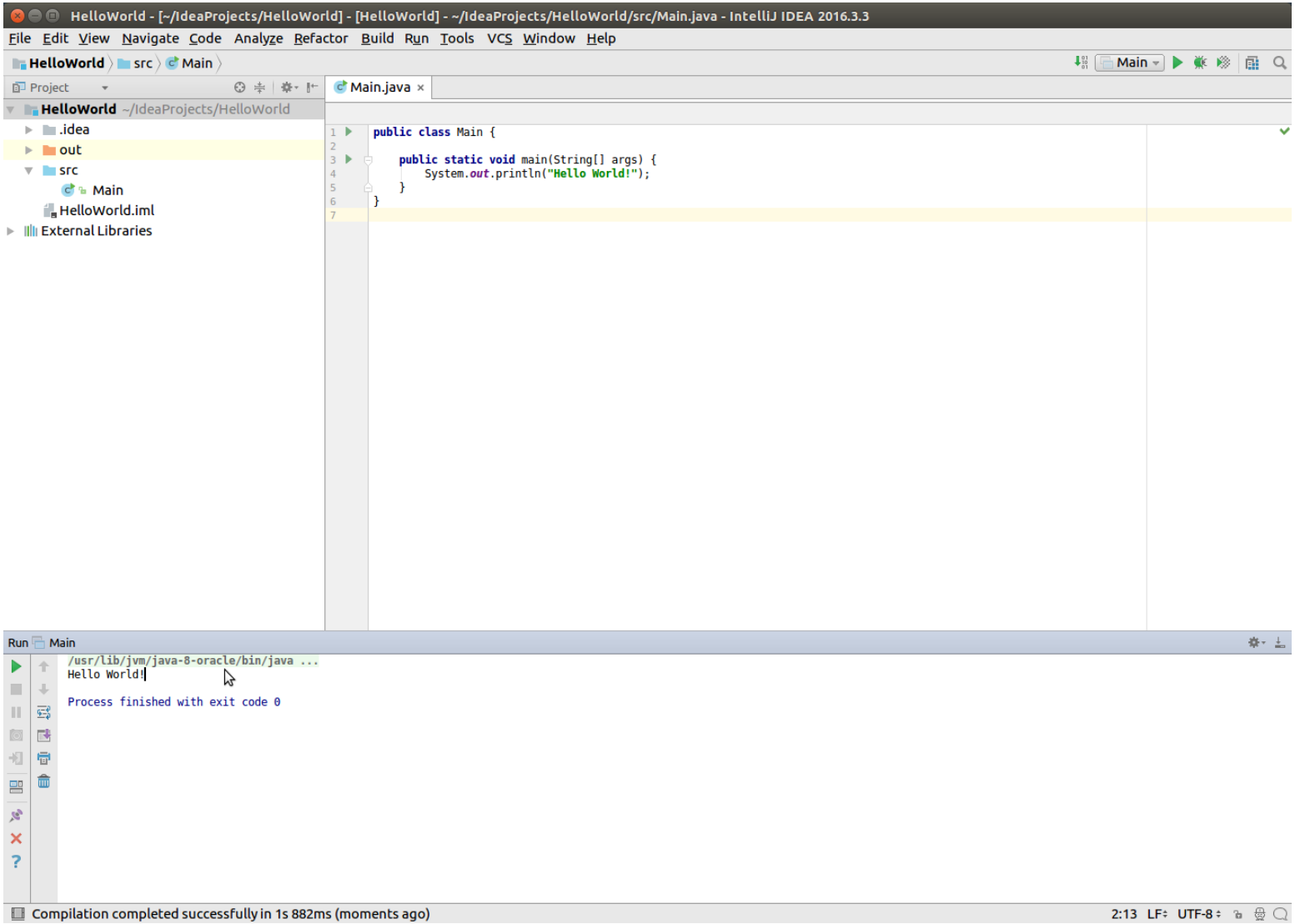
IDE


HelloWorld - [~/IdeaProjects/HelloWorld] - Main.java - IntelliJ IDEA 2016.3.3
File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
HelloWorld
Main
Project
Main.java
HelloWorld ~/IdeaProjects/HelloWorld
.idea
src
Main
HelloWorld.iml
External Libraries
Main main()
public class Main {
public static void main(String[] args) {
System.out.println("Hello World!");
}
}
Tool Windows Quick Access
Hover over the icon below to access tool windows
Click the icon to make tool windows buttons visible
Got it!
4:41 LF: UTF-8:

Running


HelloWorld - [~/IdeaProjects/HelloWorld] - [HelloWorld] - ~/IdeaProjects/HelloWorld/src/Main.java - IntelliJ IDEA 2016.3.3
File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
Run 'Main' Shift+F10
Debug 'Main' Shift+F9
Run 'Main' with Coverage
Run... Alt+Shift+F10
Debug... Alt+Shift+F9
Attach to Local Process...
Edit Configurations...
Import Test Results
Stop Ctrl+F2
Show Running List
Reload Changed Classes
Step Over F8
Force Step Over Alt+Shift+F8
Step Into F7
Force Step Into Alt+Shift+F7
Smart Step Into Shift+F7
Step Out Shift+F8
Run to Cursor Alt+F9
Force Run to Cursor Ctrl+Alt+9
Drop Frame
Pause Program
Resume Program F9
Evaluate Expression... Alt+F8
Quick Evaluate Expression Ctrl+Alt+8
Show Execution Point Alt+F10
Toggle Line Breakpoint Ctrl+F8
Toggle Method Breakpoint
Toggle Temporary Line Breakpoint Ctrl+Alt+Shift+F8
Toggle Breakpoint Enabled
View Breakpoints... Ctrl+Shift+F8
Get thread dump
Tool Windows Quick Access
Hover over the icon below to access tool windows
Click the icon to make tool windows buttons visible
Got it!
Run selected configuration
7:1 LF: UTF-8:

Execution


HelloWorld - [~/IdeaProjects/HelloWorld] - [HelloWorld] - ~/IdeaProjects/HelloWorld/src/Main.java - IntelliJ IDEA 2016.3.3
File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
HelloWorld src Main
Main
Project
Main.java
HelloWorld ~/IdeaProjects/HelloWorld
.idea
out
src
Main
HelloWorld.iml
External Libraries
public class Main {
public static void main(String[] args) {
System.out.println("Hello World!");
}
}
Run Main
/usr/lib/jvm/java-8-oracle/bin/java ...
Hello World!
Process finished with exit code 0
Compilation completed successfully in 1s 882ms (moments ago)
2:13 LF: UTF-8:

Java Program Basics

Class `HelloPrinter`

```
public class HelloPrinter
{
   public static void main(String[] arguments) {
      // Display a greeting in the console window
       System.out.println("Hello, World!");
   }
}
```

- `public class HelloPrinter`
  - Create the `HelloPrinter` class.
    - Every program consists of one or more classes.
    - Every source file can contain at most one public class, and the name of the public class must match the name of the file containing the class.
    - For example, the class `HelloPrinter` must be contained in the `HelloPrinter.java` file.
  - The reserved word `public` denotes that the class is usable by the "public".

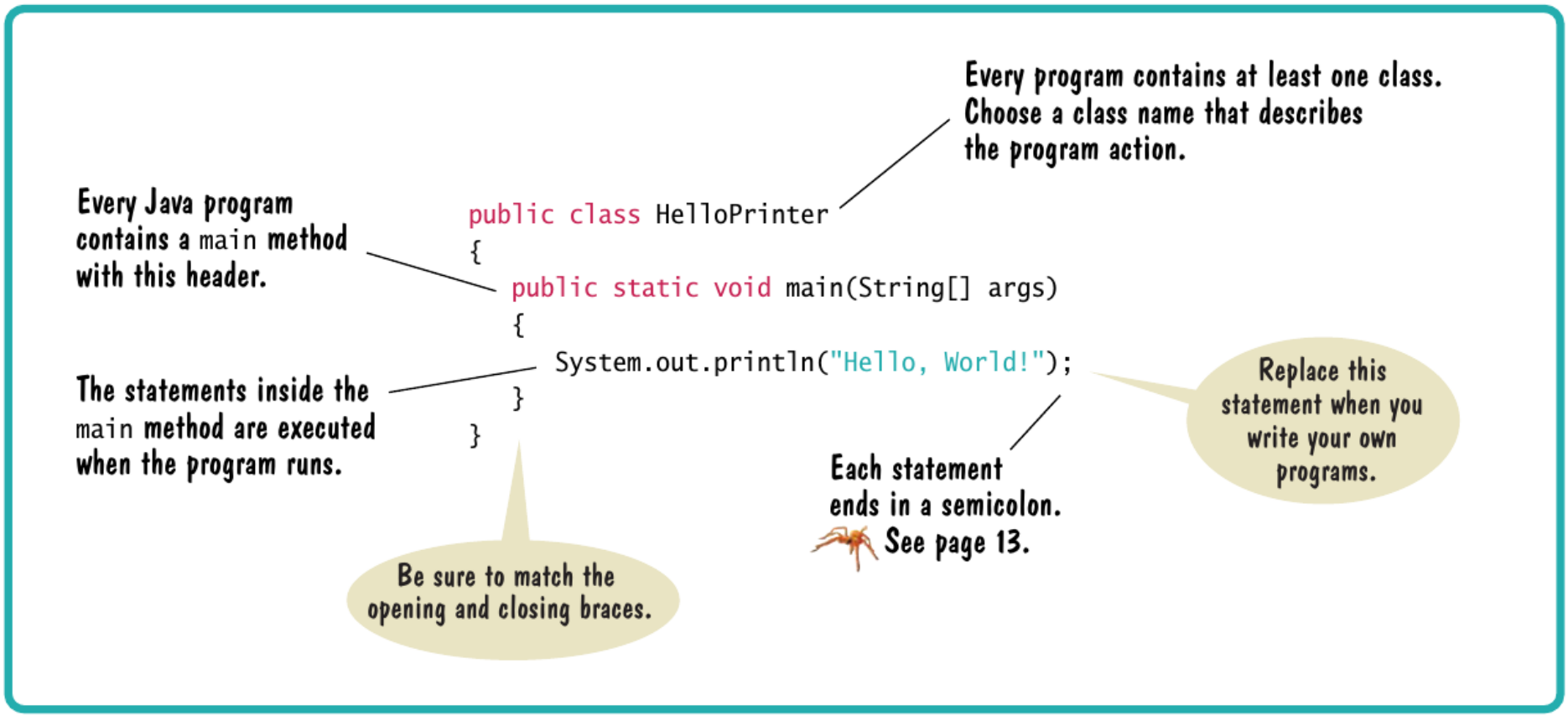
Method `main`

- `public static void main(String[] args)`
  - Every Java application must have a `main` method, which is the entry point.
  - `String[] args` contains command line arguments (passed to the `main` method.
  - The reserved word `static` indicates that the `main` method does not operate on an object.
  - `main` method must always be `static` , because it starts running before the program can create objects.
- `// Display a greeting in the console window`
  - Any text enclosed between `//` and the end of the line is completely ignored by the compiler.
  - Comments are used to explain the program to other programmers or to yourself.

Statements

- The instructions or statements in the body of the `main` method–that is, the statements inside the curly braces (`{}`) – are executed one by one.
  - Each statement ends in a semicolon (`;`).
- `System.out.println("Hello, World!");`
  - This statement prints a line of text, namely "Hello, World!".
    - The console window is represented in Java by an object called `out`, which it is placed it in the `System` class, which contains useful objects and methods to access system resources.
    - `println` method prints the received parameter, in this case the string "Hello, World!".

Simple program



Object Oriented Programming Basics

What is a class?

- `class` form the basic building blocks of any Java program.
  - Every program in Java consists of classes because the code for a program can appear only within a class definition.
- It defines a new type.
  - A class is the blueprint from which individual objects are created.
- `class` definition:
  - **Fields/instance variables**: These are variables that store data items that typically differentiate one object of the class from another. They are also referred to as data members of a class.
  - **Constructors**: They are a special type of method that is used to initialize the object. Java constructor is invoked at the time of object creation. It constructs the values i.e. provides data for the object that is why it is known as constructor.
  - **Methods**: These define the operations you can perform for the class–so they determine what you can do to, or with, objects of the class. Methods typically operate on the fields or the data members of the class.

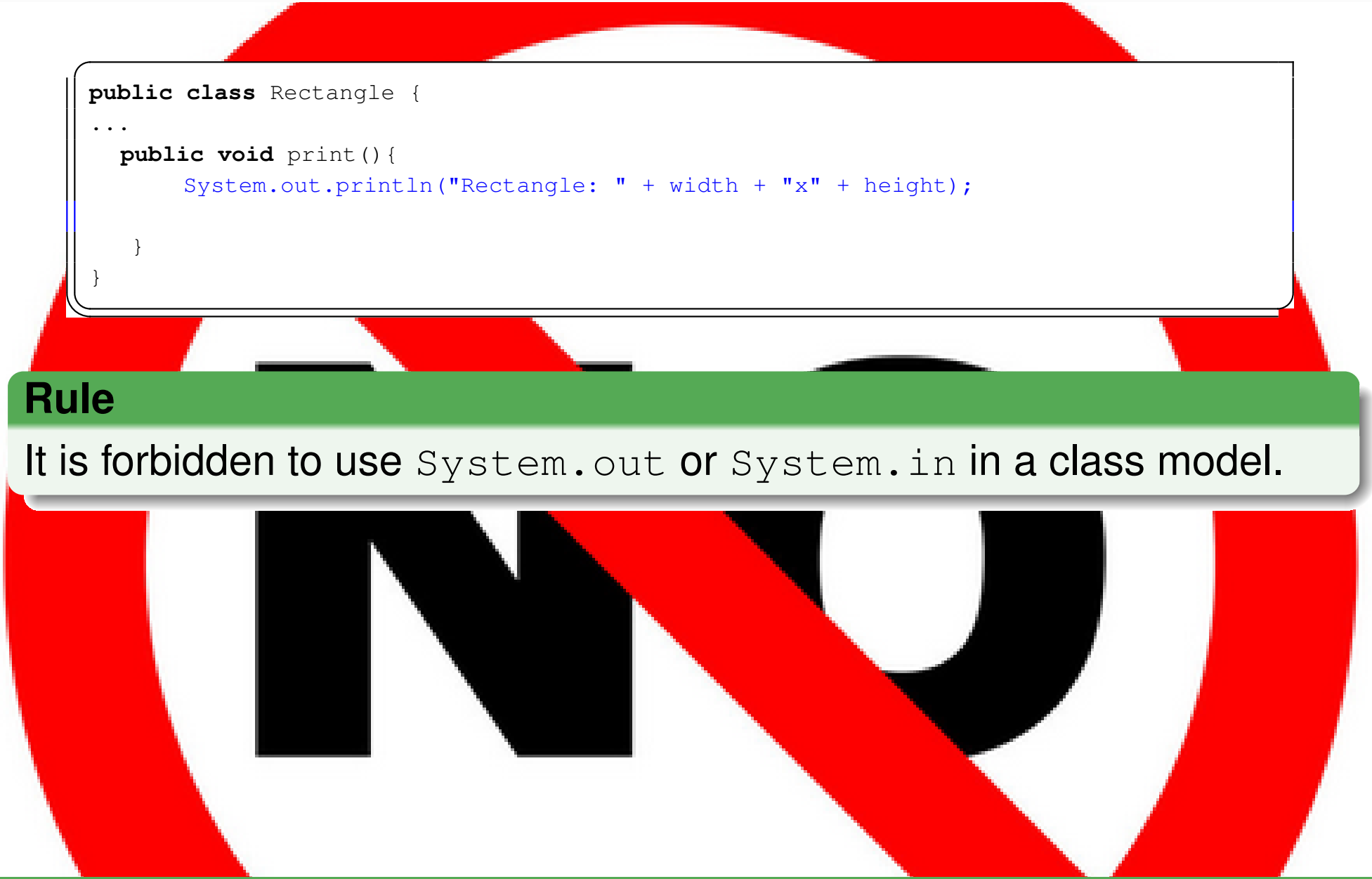
Class definition

```
public class Rectangle {
   //Variables
   private int width;
   private int height;
   //Constructors
   public Rectangle(){
      width = 0;
      height = 0;
   }
   public Rectangle(int w, int h){
      width = w;
      height = h;
   }
   //methods
   public int getArea(){
      return width * height;
   }
   public int getPerimeter(){
      return 2 * ( width + height);
   }
   public void print(){
      System.out.println("Rectangle: " + width + "x" + height);
   }
}
```

Class definition: Attention

```
public class Rectangle {
...
  public void print(){
       System.out.println("Rectangle: " + width + "x" + height);

   }
}
```



Rule

It is forbidden to use `System.out` or `System.in` in a class model.

Class definition: Solution

```
public class Rectangle {
...
   @Override
    public String toString() {
       String str="Rectangle: " + width + "x" + height;
       return str;
    }
}
```

- Use `toString` method to return the object data in a formatted string.
  - `toString` method will be addressed later.

Class variables

- A variable is a storage location in a computer program. Each variable has a name and holds a value.
- An instance variable declaration consists of the following parts:
  - An access specifier (private)
  - The type of the instance variable (such as int)
  - The name of the instance variable (such as value)

```
public class Rectangle {
   //Variables
   private int width;
   private int height;
   ...
}
```

Class constructors

- A class contains constructors that are invoked to create objects from the class blueprint.
- Constructor declarations look like method declarations–except that they use the name of the class and have no return type.

```
public class Rectangle {
   ...
   //Constructors
   public Rectangle(){
      width = 0;
      height = 0;
   }
   public Rectangle(int w, int h){
      width = w;
      height = h;
   }
...
}
```

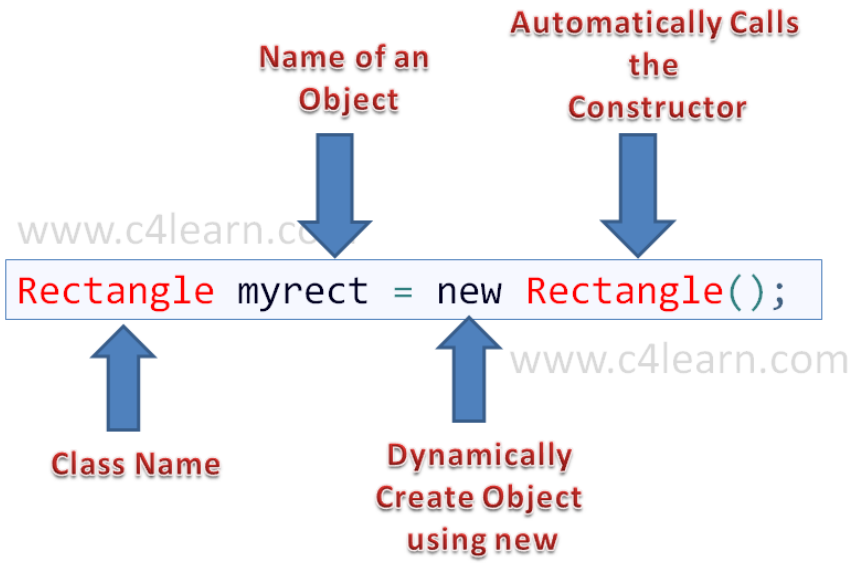
Class methods

- They require a minimum of three items:
  - **Modifier** : public, private, protected
  - **Return Type**: void, int, double, (etc.)
  - **Name**: whatever you want to call the method
  - **Parameters** (optional)

```
public class Rectangle {
   ...
    //methods
    public int getArea(){
       return width * height;
    }
    public int getPerimeter(){
       return 2 * ( width + height);
    }
}
```

What is an object?

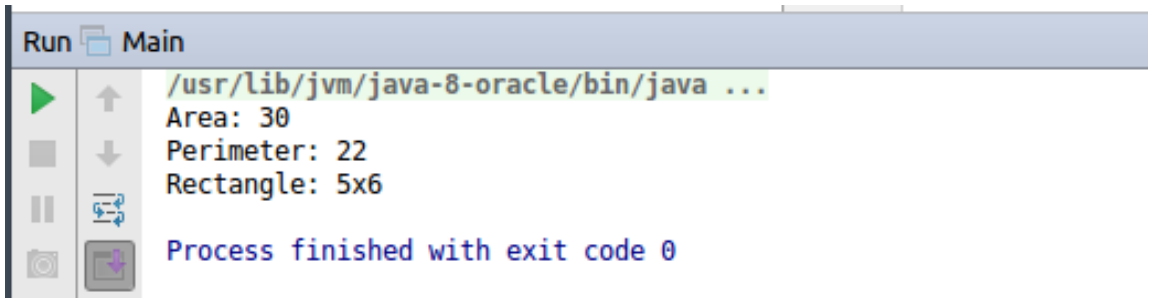
- An object is an instance of a class.



Invoking methods (I)

- Using an object instance

```
public class Main {
   public static void main(String[] args) {
  // write your code here
       Rectangle rectangle = new Rectangle(5,6);
       int area = rectangle.getArea();
       System.out.println("Area: "+ area);
       int perimeter = rectangle.getPerimeter();
       System.out.println("Perimeter: "+ perimeter);
       String str = rectangle.toString();
       System.out.println(str);
    }
}
```



Check: `TP1_01`

Invoking methods (II)

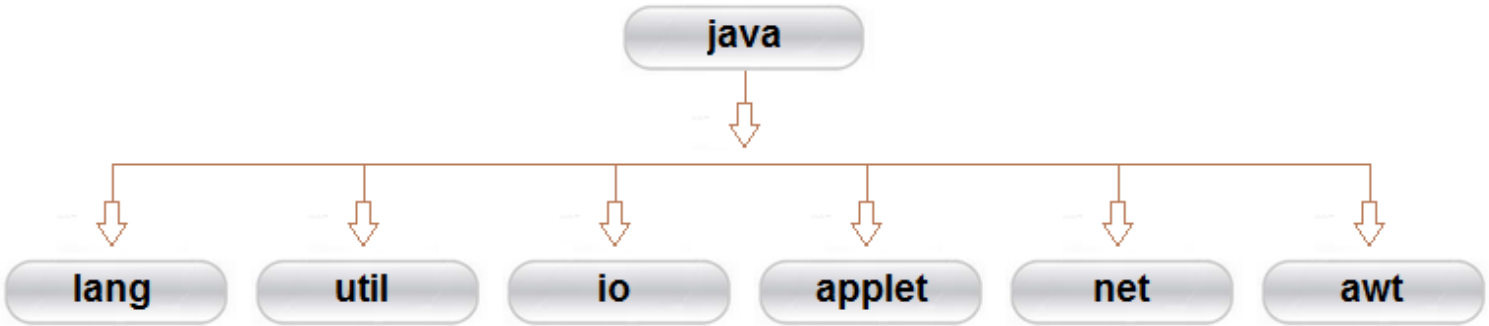
- Without an object instance
  - A `static` method can be invoked without an object instance of the class.

```
public class Main {

   static boolean isLeapYear(int year) { ..... }
   static boolean isValidDate(int year, int month, int day) { ..... }
   static int getDayOfWeek(int year, int month, int day) { ..... }
   public static void main(String[] args) {
     boolean leapYear = isLeapYear(1900);
     boolean validDate = isValidDate(2012, 2, 29);
     int dayOfWeek = getDayOfWeek(1982, 4, 24);
   }
}
```

What Is a Package?

- A package is a **namespace for organizing classes and interfaces** in a logical manner.
  - A package is a collection of classes with a related purpose.
  - Java classes are grouped into packages. Use the `import` statement to use classes that are declared in other packages.
- Conceptually you can think of **packages as being similar to different folders on your computer**.
- Java library consists of several hundred packages.



Creating a Package

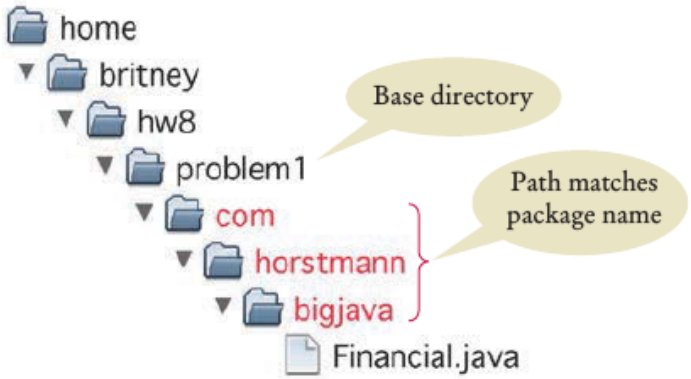
- To create a package, you choose a name for the package and put a `package` statement with that name at the top of every source file that contains the types that you want to include in the package.

Syntax `package packageName;`

```
package com.horstmann.bigjava;
```

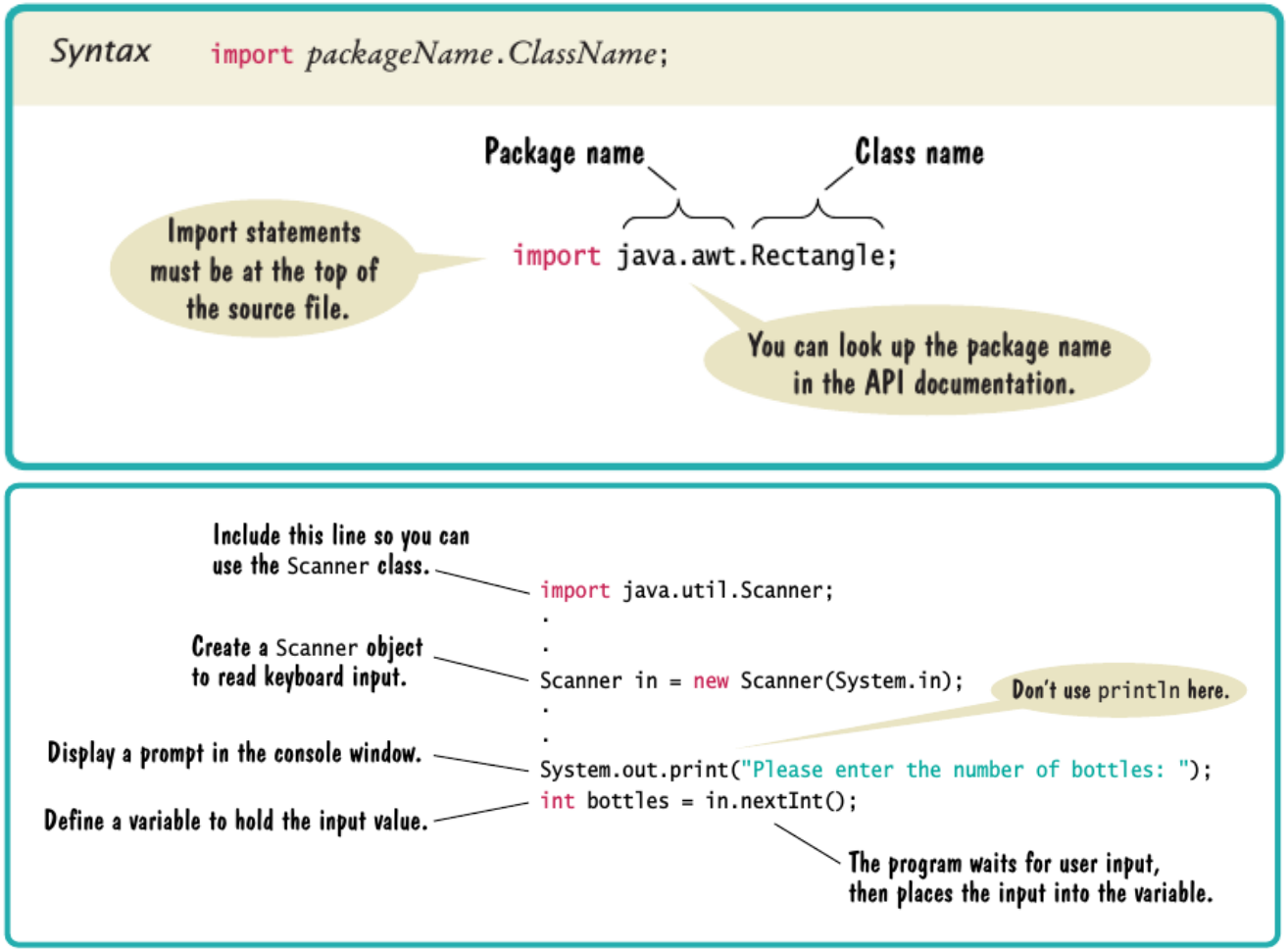
The classes in this file belong to this package.

A good choice for a package name is a domain name in reverse.



Using a Package

- Use the `import` statement to use classes that are declared in other packages.



Class

- A Java application is composed by a set of classes.

Syntax

```
accessSpecifier class ClassName
{
   instance variables
   constructors
   methods
}
```

```
public class Counter
{
   private int value;

   public Counter(int initialValue) { value = initialValue; }

   public void click() { value = value + 1; }
   public int getValue() { return value; }
}
```

Public interface

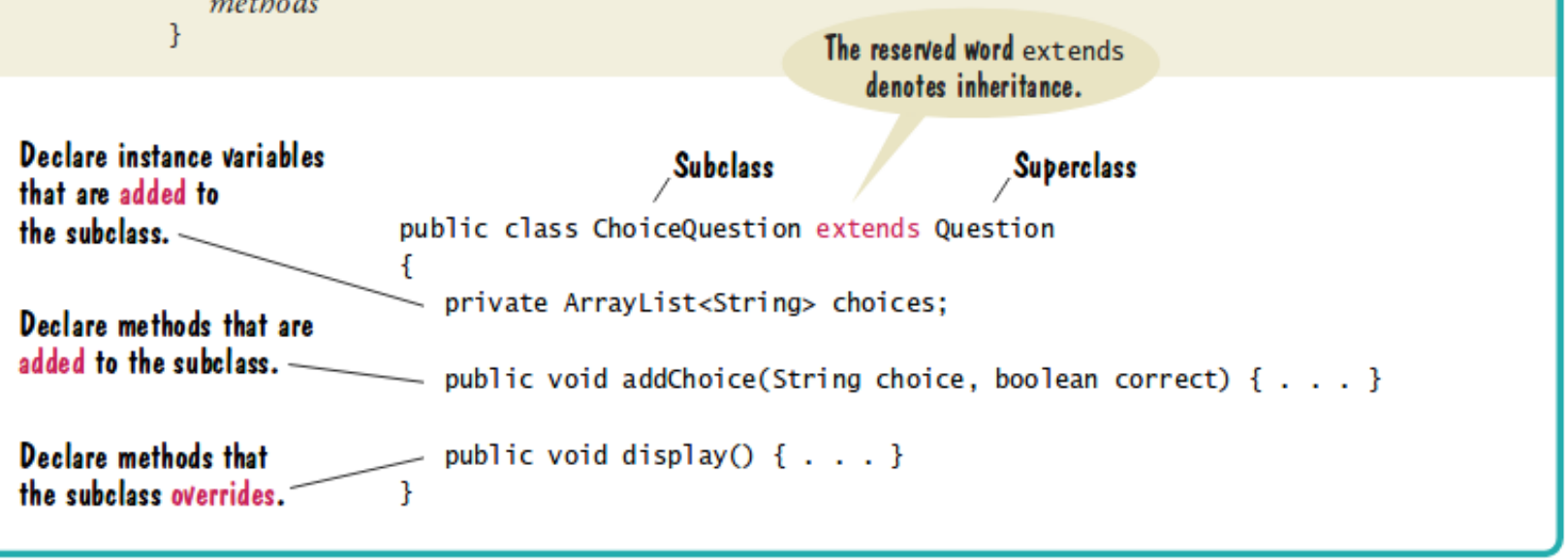
Private implementation

Subclass

- A class that inherits variables and methods from a superclass but may also add instance variables, add methods, or redefine methods.

Syntax

```
public class SubclassName extends SuperclassName
{
   instance variables
   methods
}
```



What Is an Interface? (I)

- An **interface** is a contract between a class and the outside world.
  - When a class implements an interface, it promises to provide the behavior published by that interface.
- An **interface is a group of related methods with empty bodies** (abstract).

```
interface Bicycle {
  void changeCadence(int newValue);
  void changeGear(int newValue);
  void speedUp(int increment);
  void applyBrakes(int decrement);
}
```

- Implementing an interface allows a class to become more formal about the behavior it promises to provide.

What Is an Interface? (II)

- If your class claims to implement an interface, all methods defined by that interface must appear in its source code before the class will successfully compile.

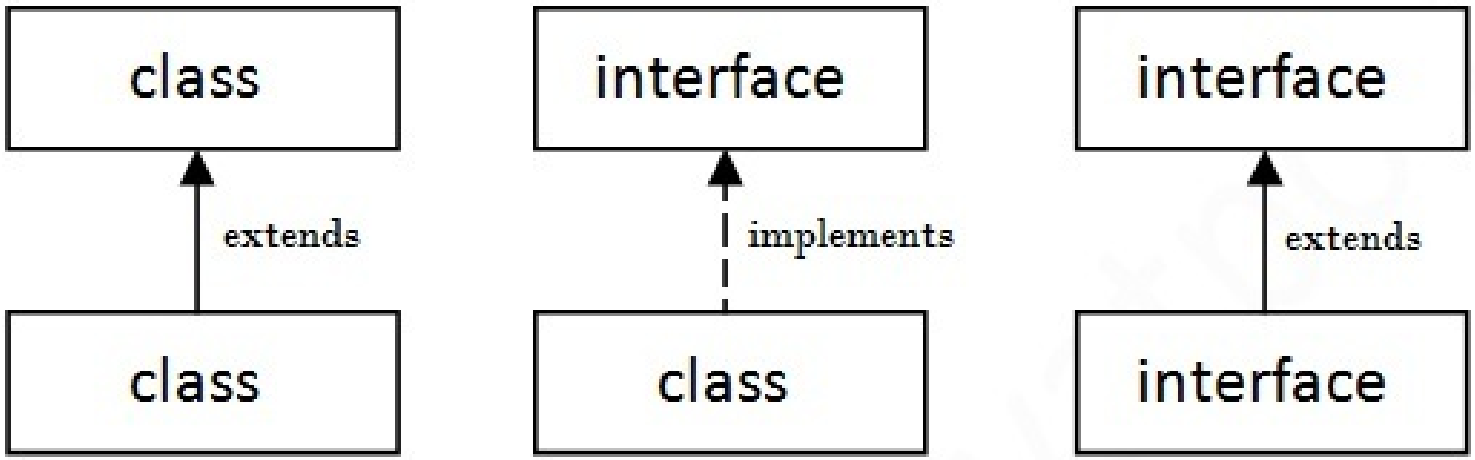
```
class MountainBike implements Bicycle {
  int cadence = 0;
  int speed = 0;
  int gear = 1;

  void changeCadence(int newValue) {
    cadence = newValue;
  }
  void changeGear(int newValue) {
    gear = newValue;
  }
  void speedUp(int increment) {
    speed = speed + increment;
  }
  void applyBrakes(int decrement) {
    speed = speed - decrement;
  }
}
```

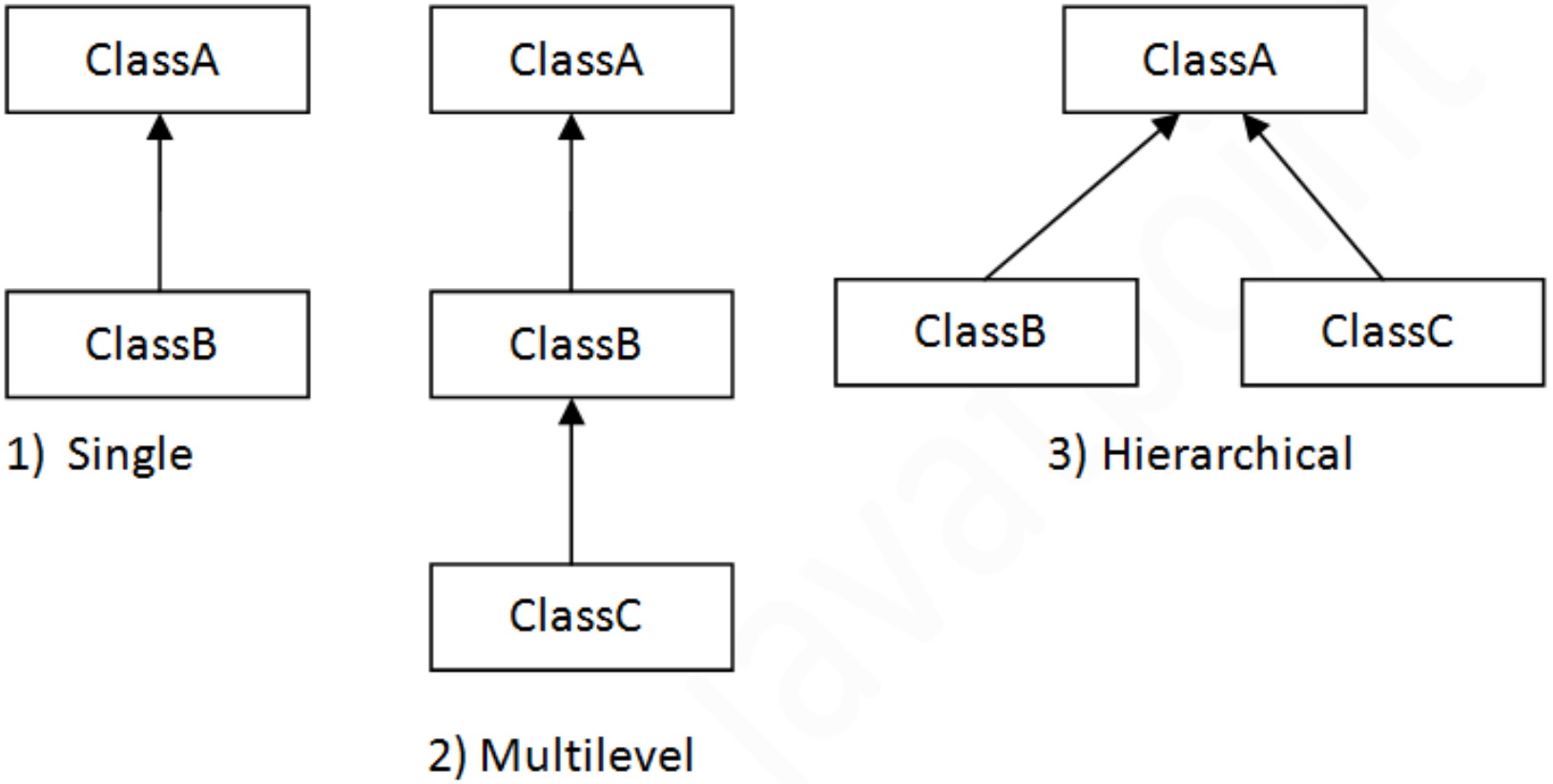
Check: `TP1_02`

Classes and Interfaces

- A class **extends** another class, an interface **extends** another interface but a class **implements** an interface.



Types of inheritance (I)



- There can be three types of inheritance: single, multilevel and hierarchical.

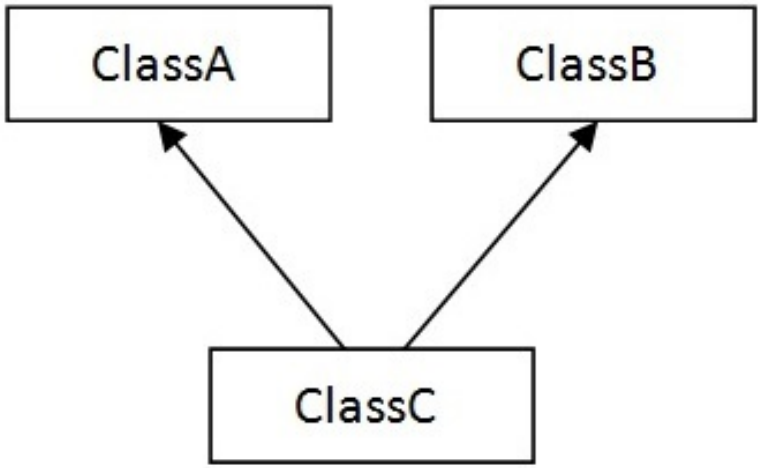
Types of inheritance (II)

```
class Animal{
  ...
}
class Dog extends Animal{
  ...
}
```

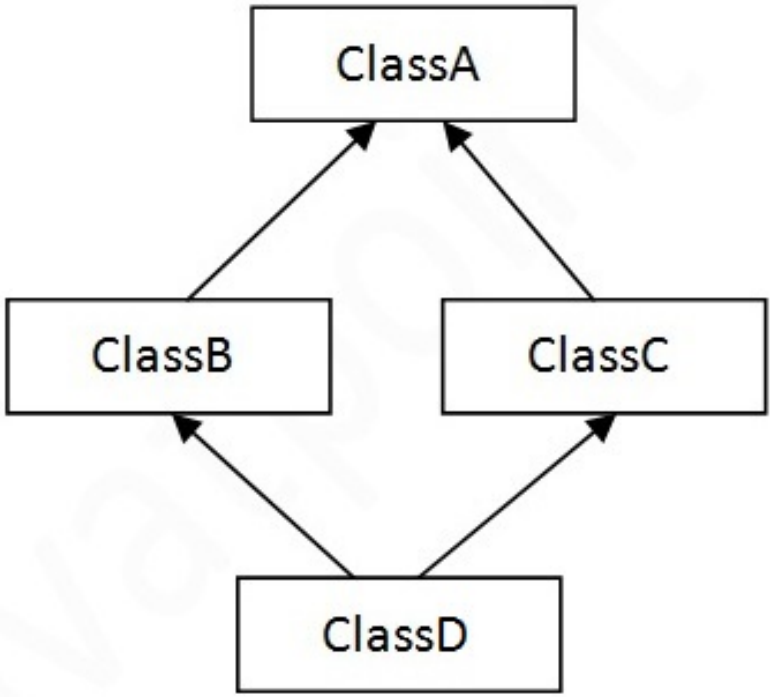
```
class Animal{
  ...
}
class Dog extends Animal{
  ...
}
class BabyDog extends Dog{
  ...
}
```

```
class Animal{
  ...
}
class Dog extends Animal{
  ...
}
class Cat extends Animal{
  ...
}
```

Types of inheritance (III)



4) Multiple



5) Hybrid

- Multiple and hybrid inheritance is supported through **interface** only

Types of inheritance (IV)

```
interface A {
  public void methodA();
}
interface B {
  public void methodB();
}
class MultipleInheritance implements A,B{
  public void methodA() {
    System.out.println("Calling methodA");
  }
  public void methodB() {
    System.out.println("Calling methodB");
  }
}
```

```
interface A {
  public void methodA();
}
interface B extends A {
  public void methodB();
}
interface C extends A {
  public void methodC();
}
class HybridInheritance implements B, C {
  public void methodA() {
    System.out.println("Calling methodA");
  }
  public void methodB() {
    System.out.println("Calling methodB");
  }
  public void methodC() {
    System.out.println("Calling methodC");
  }
}
```

Fundamental Data Types

Primitive Types

Type	Description	Size
`int`	The integer type, with range –2,147,483,648 (`Integer.MIN_VALUE`) . . . 2,147,483,647 (`Integer.MAX_VALUE`, about 2.14 billion)	4 bytes
`byte`	The type describing a single byte, with range –128 . . . 127	1 byte
`short`	The short integer type, with range –32,768 . . . 32,767	2 bytes
`long`	The long integer type, with range –9,223,372,036,854,775,808 . . . 9,223,372,036,854,775,807	8 bytes
`double`	The double-precision floating-point type, with a range of about $\pm 10^{308}$ and about 15 significant decimal digits	8 bytes
`float`	The single-precision floating-point type, with a range of about $\pm 10^{38}$ and about 7 significant decimal digits	4 bytes
`char`	The character type, representing code units in the Unicode encoding scheme (see Computing & Society 4.2 on page 161)	2 bytes
`boolean`	The type with the two truth values `false` and `true` (see Chapter 5)	1 bit

Literal

- A **literal** is the source code representation of a fixed value.
  - Literals in Java are a sequence of characters (digits, letters, and other characters) that represent constant values to be stored in variables.

Number	Type	Comment
6	int	An integer has no fractional part.
-6	int	Integers can be negative.
0	int	Zero is an integer.
0.5	double	A number with a fractional part has type double.
1.0	double	An integer with a fractional part .0 has type double.
1E6	double	A number in exponential notation: 1×10^6 or 1000000. Numbers in exponential notation always have type double.
2.96E-2	double	Negative exponent: $2.96 \times 10^{-2} = 2.96 / 100 = 0.0296$
100000L	long	The L suffix indicates a long literal.
100,000		**Error:** Do not use a comma as a decimal separator.
100_000	int	You can use underscores in number literals.
3 1/2		**Error:** Do not use fractions; use decimal notation: 3.5

Assignment

- =
  - It assigns the value on its right to the operand on its left

```
public static void main (String[] args) {
      int result = 1;
}
```

Aritmetic

- +
  - Additive operator (also used for String concatenation)
- -
  - Subtraction operator
- *
  - Multiplication operator
- /
  - Division operator
- %
  - Remainder operator

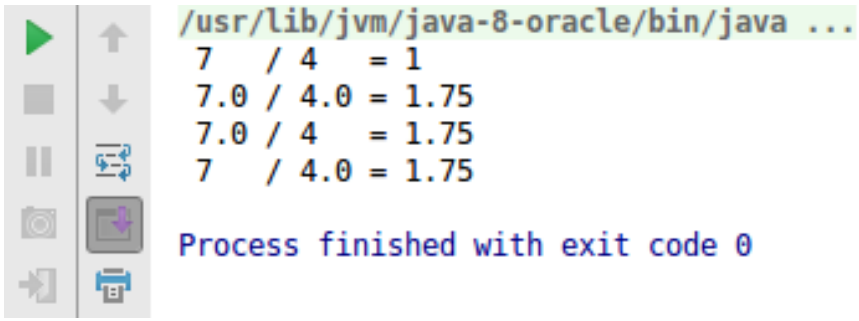
```
public static void main (String[] args) {
      int result = 1 + 2;
      // result is now 3
      System.out.println("1 + 2 = " + result);
}
```

Check: `TP1_03`

Aritmetic: division

- If both arguments of a division (/) are integers, the remainder is discarded.
- If at least one is floating-point number the remainder is not discarded.

```
public static void main (String[] args) {
      System.out.println(" 7 / 4 = " + 7/4);
      System.out.println(" 7.0 / 4.0 = " + 7.0/4.0);
      System.out.println(" 7.0 / 4 = " + 7.0/4);
      System.out.println(" 7 / 4.0 = " + 7/4.0);
}
```



Unary

- +
  - Unary plus operator; indicates positive value (numbers are positive without this, however)
- -
  - Unary minus operator; negates an expression
- ++
  - Increment operator; increments a value by 1
- - -
  - Decrement operator; decrements a value by 1
- !
  - Logical complement operator; inverts the value of a boolean

Check: `TP1_04`

Unary: Increment/decrement operators

- The increment/decrement operators can be applied before (prefix) or after (postfix) the operand.
  - The code `result++;` and `++result;` will both end in result being incremented by one.
  - The only difference is that the prefix version (`++result;`) evaluates to the incremented value, whereas the postfix version (`result++;`) evaluates to the original value.

Check: `TP1_05`

Powers, roots and others

- In Java, there are no symbols for powers and roots. You have to use the Mathematical Java library.

Method	Returns	Method	Returns
`Math.sqrt(x)`	Square root of x (≥ 0)	`Math.abs(x)`	Absolute value $\lvert x \rvert$
`Math.pow(x, y)`	x^y ($x > 0$, or $x = 0$ and $y > 0$, or $x < 0$ and y is an integer)	`Math.max(x, y)`	The larger of x and y
`Math.sin(x)`	Sine of x (x in radians)	`Math.min(x, y)`	The smaller of x and y
`Math.cos(x)`	Cosine of x	`Math.exp(x)`	e^x
`Math.tan(x)`	Tangent of x	`Math.log(x)`	Natural log ($\ln(x)$, $x > 0$)
`Math.round(x)`	Closest integer to x (as a `long`)	`Math.log10(x)`	Decimal log ($\log_{10}(x)$, $x > 0$)
`Math.ceil(x)`	Smallest integer $\geq x$ (as a `double`)	`Math.floor(x)`	Largest integer $\leq x$ (as a `double`)
`Math.toRadians(x)`	Convert x degrees to radians (i.e., returns $x \cdot \pi / 180$)	`Math.toDegrees(x)`	Convert x radians to degrees (i.e., returns $x \cdot 180 / \pi$)

Check: `TP1_06`

Arithmetic expressions

Mathematical Expression	Java Expression	Comments
$\frac{x+y}{2}$	`(x + y) / 2`	The parentheses are required; `x + y / 2` computes $x + \frac{y}{2}$.
$\frac{xy}{2}$	`x * y / 2`	Parentheses are not required; operators with the same precedence are evaluated left to right.
$\left(1+\frac{r}{100}\right)^n$	`Math.pow(1 + r / 100, n)`	Use `Math.pow(x, n)` to compute x^n.
$\sqrt{a^2+b^2}$	`Math.sqrt(a * a + b * b)`	`a * a` is simpler than `Math.pow(a, 2)`.
$\frac{i+j+k}{3}$	`(i + j + k) / 3.0`	If i, j, and k are integers, using a denominator of 3.0 forces floating-point division.
π	`Math.PI`	`Math.PI` is a constant declared in the `Math` class.

Cast

- You use a cast (typeName) to convert a value to a different type.
  - You must use the cast operator (int) to convert a convert floating-point value to an integer.
  - Write the cast operator before the expression that you want to convert:

```
double balance = total + tax;
int dollars = (int) balance;
```

  - The cast `(int)` converts the floating-point value `balance` to an integer by discarding the fractional part. For example, if `balance` is 13.75, then `dollars` is set to 13.

Control Statements

Relational Operators

- A relational operator tests the relationship between two values.

Java	Math Notation	Description
>	>	Greater than
>=	≥	Greater than or equal
<	<	Less than
<=	≤	Less than or equal
==	=	Equal
!=	≠	Not equal

Conditional Operators

- `&&` and `||` operators perform Logical AND and Logical OR operations on two boolean expressions.
- `!` operator perform Logical NOT operation of a boolean expression.

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

A	B	A || B
true	true	true
true	false	true
false	true	true
false	false	false

A	!A
true	false
false	true

The `if` statement

Syntax

```
if (condition)
{
   statements
}
```

```
if (condition) { statements1 }
else { statements2 }
```

A condition that is true or false.
Often uses relational operators:
== != < <= > >= (See page 184.)

Braces are not required if the branch contains a single statement, but it's good to always use them. See page 181.

```
if (floor > 13)
{
   actualFloor = floor - 1;
}
else
{
   actualFloor = floor;
}
```

Don't put a semicolon here! See page 182.

If the condition is true, the statement(s) in this branch are executed in sequence; if the condition is false, they are skipped.

Omit the `else` branch if there is nothing to do.

Lining up braces is a good idea. See page 181.

If the condition is false, the statement(s) in this branch are executed in sequence; if the condition is true, they are skipped.

Check: `TP1_07`

The ternary Operator

- Java has a conditional operator of the form:

```
condition ? value1 : value2
```

  - The value of that expression is either `value1` if the test passes or `value2` if it fails.
  - For example, we can compute the actual floor number as:

```
actualFloor = floor > 13 ? floor - 1 : floor;
```

    which is equivalent to to.

```
if (floor > 13) {
   actualFloor = floor - 1;
} else {
   actualFloor = floor;
}
```

The `switch` statement

```
switch (expression) {
  case value1:
  // statement sequence
  break;
  case value2:
  // statement sequence
  break;
  ...
  case valueN:
  // statement sequence
  break;
  default:
  // default statement sequence
}
```

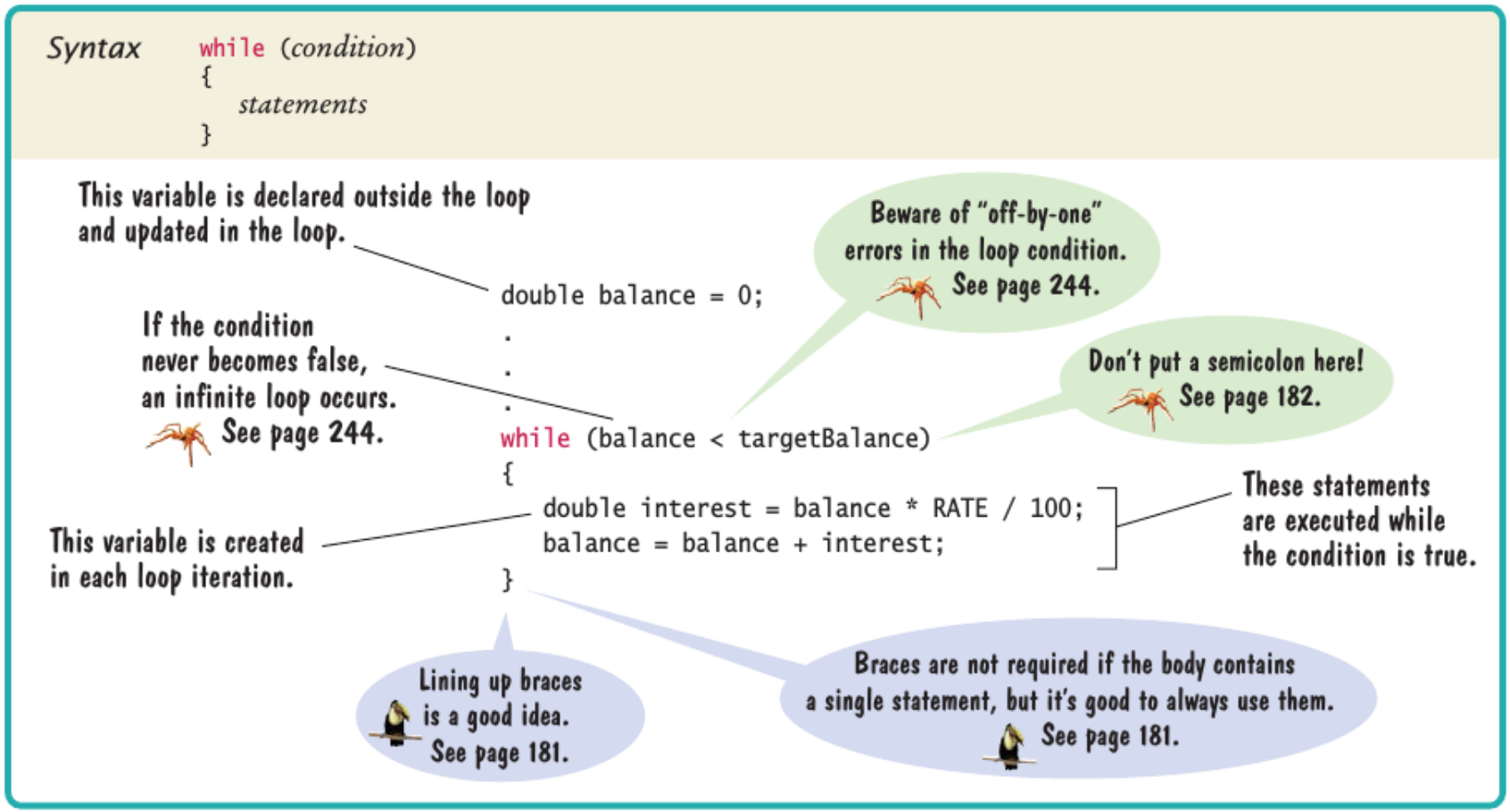
- The expression must be of type `byte`, `short`, `int`, or `char`; each of the values specified in the case statements must be of a type compatible with the expression.

Check: `TP1_08`

The `while` loop

Syntax

```
while (condition)
{
   statements
}
```



The `do - while` loop

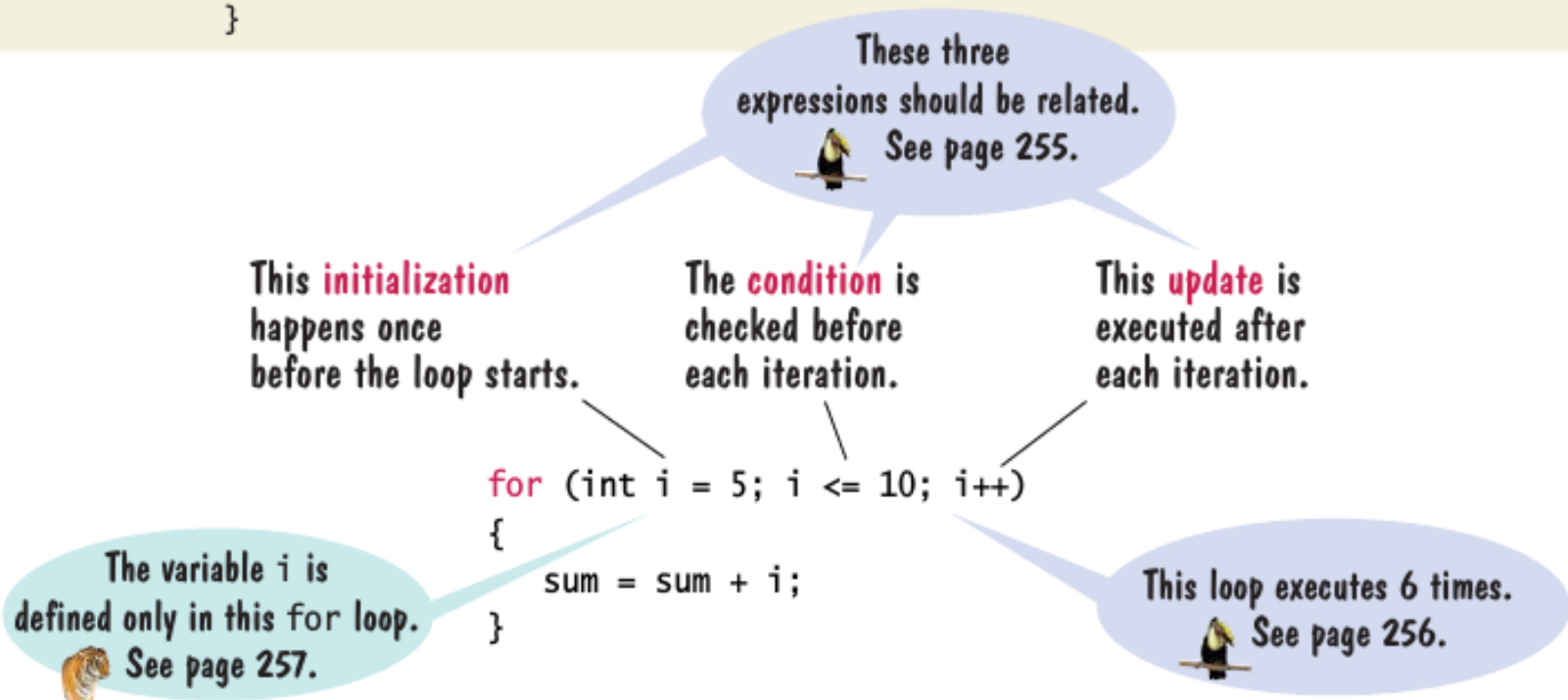
- `do - while` loop is similar to `while` loop, however there is a single difference between these two.
- Unlike `while` loop, `do - while` guarantees at-least one execution of block of statements.
  - This happens because the `do - while` loop evaluates the boolean expression at the end of the loop?s body.
  - Therefore the set of statements gets executed at-least once before the check of boolean expression.

```
do {
    statement(s)
} while (expression);
```

The `for` loop

Syntax

```
for (initialization; condition; update)
{
   statements
}
```



The `break` Statement

- The `break` statement is used in the `switch` statement..
- You can also use the `break` statement to terminate a `for`, `while`, or `do-while` loop.

```
for(int i=0; i<100; i++) {
   if(i == 10)
      break;
}
```

- A `break` statement terminates the innermost `switch`, `for`, `while`, or `do-while` statement

```
for(int i=0; i<3; i++) {
   for(int j=0; j<100; j++) {
      if(j == 10)
         break;
   }
}
```

Check: `TP1_09`

The `continue` Statement

- Sometimes it is useful to force an early iteration of a loop. That is, you might want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration.
  - This is, in effect, a `goto` just past the body of the loop, to the loop's end. The `continue` statement performs such an action.
  - In `while` and `do-while` loops, a `continue` statement causes control to be transferred directly to the conditional expression that controls the loop.
  - In a `for` loop, control goes first to the iteration portion of the for statement and then to the conditional expression. For all three loops, any intermediate code is bypassed.

```
for(int i=0; i<10; i++) {
   System.out.print(i + " ");
   if (i%2 == 0)
      continue;
   System.out.println("");
}
```

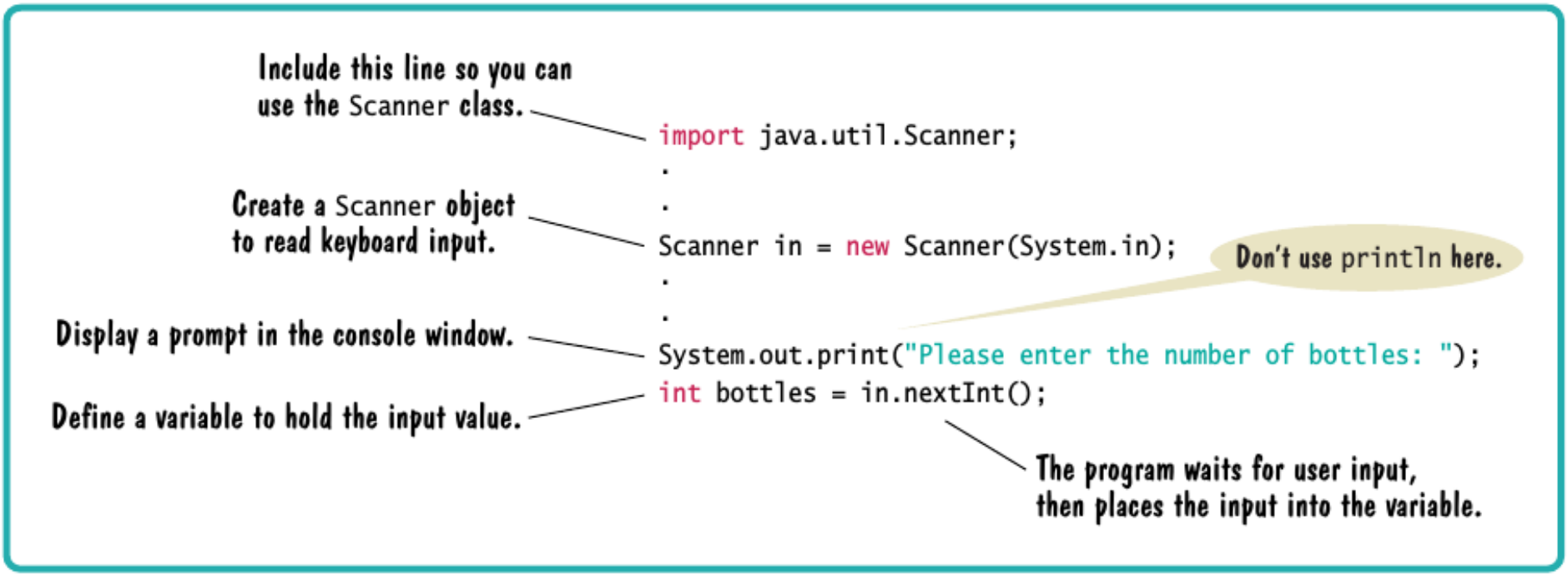
The `return` Statement

- The `return` statement is used to explicitly return from a method. That is, it causes program control to transfer back to the caller of the method.

```
public static void main(String args[]) {
   boolean t = true;
   System.out.println("Before the return.");
   if(t)
      return;
   System.out.println("This won't execute.");
}
```

Input and Output

The Scanner


Include this line so you can use the Scanner class.
import java.util.Scanner;
Create a Scanner object to read keyboard input.
Scanner in = new Scanner(System.in);
Don't use println here.
Display a prompt in the console window.
System.out.print("Please enter the number of bottles: ");
Define a variable to hold the input value.
int bottles = in.nextInt();
The program waits for user input, then places the input into the variable.

Formatted output

- Use the `printf` method (of the `System.out` to specify how values should be formatted.

```
System.out.printf("Quantity: %d Total: %10.2f", quantity, total);
```

Format String	Sample Output	Comments
"%d"	24	Use d with an integer.
"%5d"	24	Spaces are added so that the field width is 5.
"Quantity:%5d"	Quantity: 24	Characters inside a format string but outside a format specifier appear in the output.
"%f"	1.21997	Use f with a floating-point number.
"%.2f"	1.22	Prints two digits after the decimal point.
"%7.2f"	1.22	Spaces are added so that the field width is 7.
"%s"	Hello	Use s with a string.
"%d %.2f"	24 1.22	You can format multiple values at once.

Check: `TP1_10`

Strings

What is a string?

- Strings are sequences of characters.
- `String` type.

```
String name = "Harry";
```

  - `name` is a string variable.
  - `"Harry"` is a string literal.
- The number of characters in a string is called the **length** of the string.

```
int n = name.length();
```

  - A string of length 0 is called the **empty** string.
    - It contains no characters and is written as "".

```
String name = "";
```

- String positions are counted starting with 0.



Creating a String

- `String` is a Java class.
- There are two ways to create a `String` objects:
  - String literal

```
String str1 = "Welcome";
```

    - String object is created by compiler.
  - Using `new` keyword.

```
String str1 = new String("Welcome");
```

Concatenation

- Use the + operator to concatenate strings; that is, to put them together to yield a longer string.

```
String fName = new String("Harry");
String lName = new String("Morgan");
String name = new String(fName + " " + lName);
```

- Whenever one of the arguments of the + operator is a string, the other argument is converted to a string.

```
String jobTitle = new String("Agent");
int employeeId = 7;
String bond = new String(jobTitle + employeeId);
```

Reading a string

- Use the `next` method of the `Scanner` class to read a string containing a single word.

```
System.out.print("Please enter one word: ");
String word = new String(in.next());
```

- Use the `nextLine` method of the `Scanner` class to read characters until it finds a new line character `'\n'`.

```
System.out.print("Please write a sentence: ");
String sentence = new String(in.nextLine());
```

Escape Sequences

- To include a quotation mark in a literal string, precede it with a backslash (\), like this:

```
String message = new String("He said \"Hello\"");
```

  - The backslash is not included in the string. It indicates that the quotation mark that follows should be a part of the string and not mark the end of the string.
  - The sequence \" is called an escape sequence.
- To include a backslash in a string, use the escape sequence \\, like this:

```
String path = new String("C:\\Temp\\Secret.txt");
```

Strings and Characters

- Strings are sequences of Unicode characters.
- A character is a value of the type `char`.
  - Characters have numeric values
    - Character `'H'` is number 72.
- Character literals are delimited by single quotes (' '), while strings are delimited by quotes (" ");
  - `'H'` is a character, a value of type `char`.
  - `"H"` is a string containing a single character, a value of type `String`.
- The `charAt` method returns a char value from a string.

```
String name = new String("Harry");
char start = name.charAt(0);
char last = name.charAt(4);
```

  - It sets `start` to the value `'H'` and `last` to the value `'y'`.

Substrings

- Use the `substring` method to extract a part of a string.
  - The method call `str.substring(start, pastEnd)`
    - It returns a string that is made up of the characters in the string `str`, starting at position `start`, and containing all characters up to, but not including, the position `pastEnd`.

```
String greeting = new String("Hello, World!");
String sub = greeting.substring(0, 5); // sub is "Hello"
```



Comparing strings

- `boolean equals(Object obj)`: Compares the string with the specified string and returns true if both matches else false.

```
String str1= new String("Hello");
String str2= new String("Hi");
if(str1.equals(str2)){
  //equals
}else{
  //different
}
```

- `int compareTo(String string)`: This method compares the two strings based on the Unicode value of each character in the strings.

```
    String str1= new String("Hello");
String str2= new String("Hi");
String str3 = "Hello";
int var1 = str1.compareTo( str2 );
System.out.println(str1 +" & " + str2 +" comparison: "+var1);
int var2 = str1.compareTo( str3 );
System.out.println(str1 +" & " + str3 +" comparison: "+var2);
```

Converting strings

- `int parseInt(String s)`: convert the string `s` to integer

```
String str3=new String("1234");
int num3 = Integer.parseInt(str3);
```

- `String valueOf(data type)`: This method returns a string representation of specified data type.

```
int ivar = 150;
String str = new String(String.valueOf(ivar));
```

Check: `TP1_11`

String Operations

Statement	Result	Comment
`string str = "Ja";` `str = str + "va";`	`str` is set to `"Java"`	When applied to strings, + denotes concatenation.
`System.out.println("Please"` `  + " enter your name: ");`	Prints `Please enter your name:`	Use concatenation to break up strings that don't fit into one line.
`team = 49 + "ers"`	`team` is set to `"49ers"`	Because `"ers"` is a string, 49 is converted to a string.

`String first = in.next();`
`String last = in.next();`
`(User input: Harry Morgan)`
	`first` contains `"Harry"`
	`last` contains `"Morgan"`
		The `next` method places the next word into the string variable.

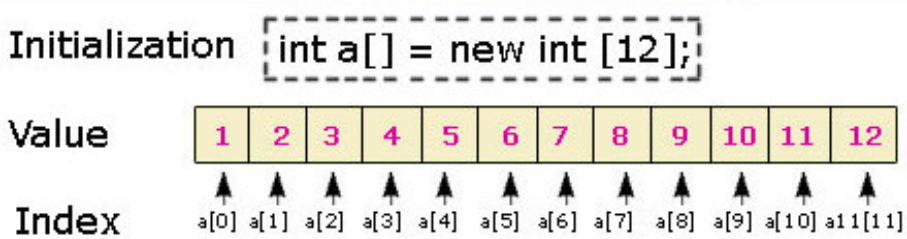
`String greeting = "H & S";` `int n = greeting.length();`	`n` is set to 5	Each space counts as one character.
`String str = "Sally";` `char ch = str.charAt(1);`	`ch` is set to `'a'`	This is a `char` value, not a `String`. Note that the initial position is 0.
`String str = "Sally";` `String str2 = str.substring(1, 4);`	`str2` is set to `"all"`	Extracts the substring starting at position 1 and ending before position 4.
`String str = "Sally";` `String str2 = str.substring(1);`	`str2` is set to `"ally"`	If you omit the end position, all characters from the position until the end of the string are included.
`String str = "Sally";` `String str2 = str.substring(1, 2);`	`str2` is set to `"a"`	Extracts a `String` of length 1; contrast with `str.charAt(1)`.
`String last = str.substring(` `  str.length() - 1);`	`last` is set to the string containing the last character in `str`	The last character has position `str.length() - 1`.

Check: `http://beginnersbook.com/2013/12/java-strings/`

Arrays

What is an Array?

- An array is a container object that holds a fixed number of values of a single type.
- The length of an array is established when the array is created.
  - After creation, its length is fixed.
- Each item in an array is called an element, and each element is accessed by its numerical index.
- The elements of arrays are numbered starting at 0.



  - The expression `a.length` yields the length of the values array. Note that there are no parentheses following length.

Declaring and Accessing (I)

Syntax

To construct an array: `new typeName[length]`

To access an element: `arrayReference[index]`

Name of array variable

Element type

Length

Type of array variable

```
double[] values = new double[10];
double[] moreValues = { 32, 54, 67.5, 29, 35 };
```

List of initial values

Use brackets to access an element.

```
values[i] = 0;
```

The index must be ≥ 0 and < the length of the array.



See page 314.

- An array index must be at least zero and less than the length of the array.
- A bounds error, which occurs if you supply an invalid array index, can cause your program to terminate.

Declaring and Accessing (II)

`int[] numbers = new int[10];`	An array of ten integers. All elements are initialized with zero.
`final int LENGTH = 10;` `int[] numbers = new int[LENGTH];`	It is a good idea to use a named constant instead of a "magic number".
`int length = in.nextInt();` `double[] data = new double[length];`	The length need not be a constant.
`int[] squares = { 0, 1, 4, 9, 16 };`	An array of five integers, with initial values.
`String[] friends = { "Emily", "Bob", "Cindy" };`	An array of three strings.
 `double[] data = new int[10];`	**Error:** You cannot initialize a `double[]` variable with an array of type `int[]`.

```
  int[] a = new int [12];
  for (int i = 0; i<a.length; i++ )
  a[i] = i + 1;
  for (int i = 0; i<a.length; i++ )
  System.out.println(a[i]);
}
```

Array Lists

What is an Array List?

- It is a class.
- It is a **container** that store lists of objects.
- It offers two significant advantages:
  - It can grow and shrink as needed.
  - The `ArrayList` class supplies methods for common tasks, such as inserting and removing elements.
- The `ArrayList` class is a generic class: `ArrayList<Type>` collects elements of the specified type.
  - The `Type` is a class.

```
ArrayList<String> names = new ArrayList<String>();
names.add("Emily");
names.add("Bob");
names.add("Cindy");

for (int i = 0; i < names.size(); i++) {
  System.out.println(names.get(i));
}
```

Declaring

Syntax

To construct an array list: `new ArrayList<typeName>()`

To access an element: *arraylistReference*`.get(index)`
arraylistReference`.set(index, value)`

Variable type — Variable name — An array list object of size 0

```
ArrayList<String> friends = new ArrayList<String>();
```

```
friends.add("Cindy");
String name = friends.get(i);
friends.set(i, "Harry");
```

The add method appends an element to the array list, increasing its size.

Use the get and set methods to access an element.

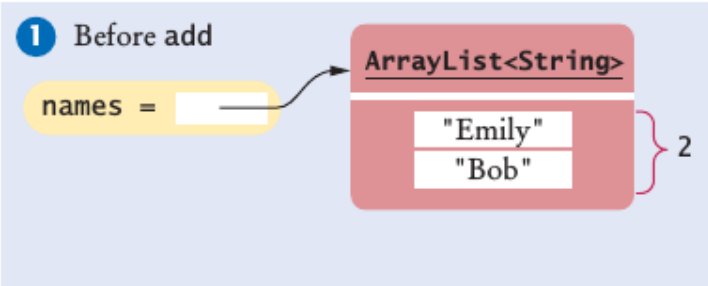
The index must be ≥ 0 and < friends.size().

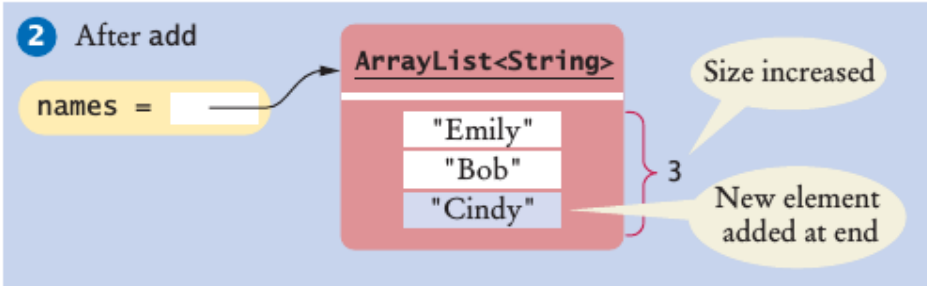
Check: `http://beginnersbook.com/2013/12/java-arraylist/`

Operations (I)

```
ArrayList<String> names = new ArrayList<String>();
names.add("Emily");
names.add("Bob");
```

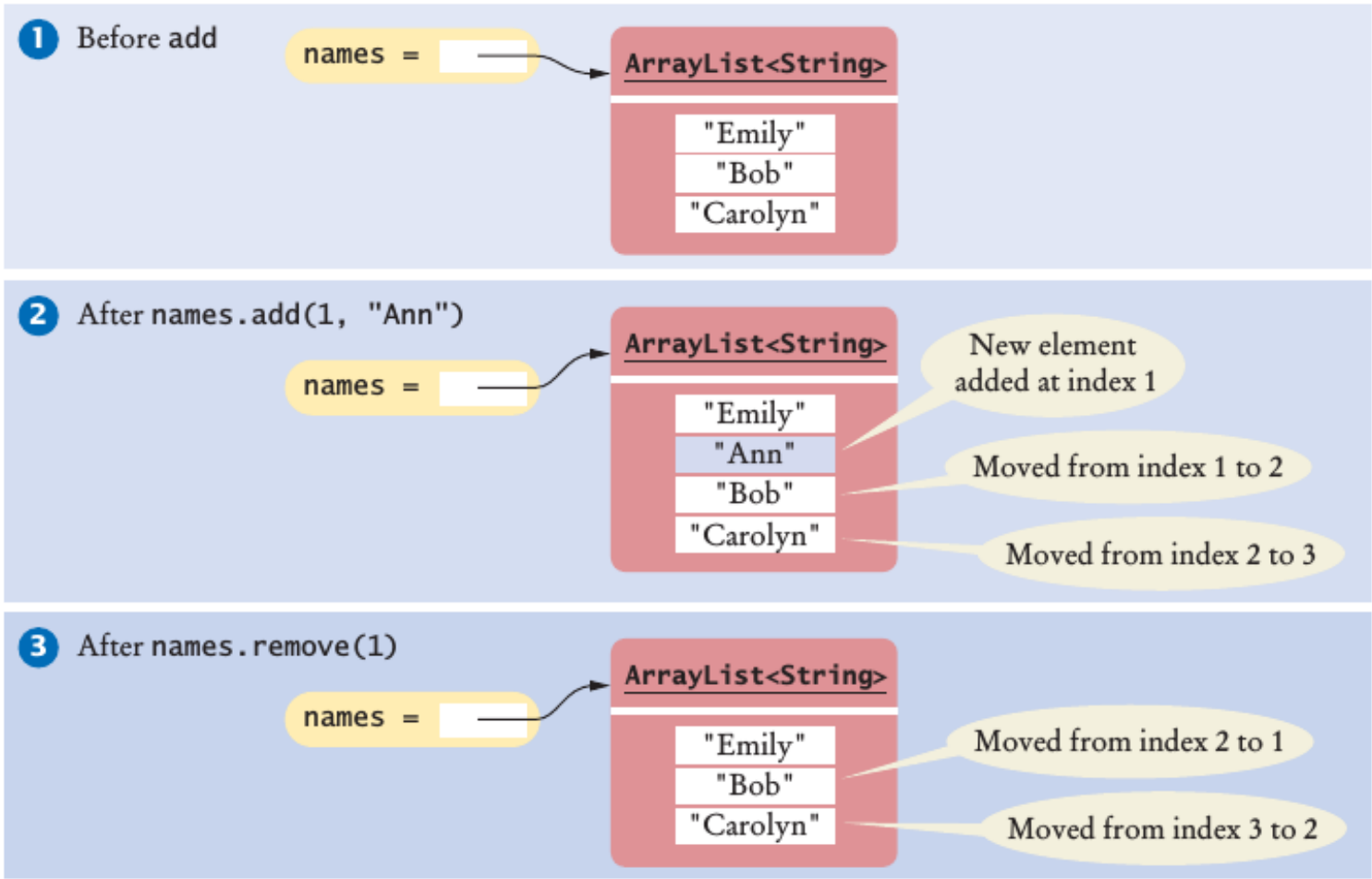
```
names.add("Cindy");
names.set(2, "Carolyn");
```





Operations (II)

```
names.add(1, "Ann");
names.remove(1);
```



Working with Array Lists

`ArrayList<String> names = new ArrayList<String>();`	Constructs an empty array list that can hold strings.
`names.add("Ann");` `names.add("Cindy");`	Adds elements to the end of the array list.
`System.out.println(names);`	Prints `[Ann, Cindy]`.
`names.add(1, "Bob");`	Inserts an element at index 1. `names` is now `[Ann, Bob, Cindy]`.
`names.remove(0);`	Removes the element at index 0. `names` is now `[Bob, Cindy]`.
`names.set(0, "Bill");`	Replaces an element with a different value. `names` is now `[Bill, Cindy]`.
`String name = names.get(i);`	Gets an element.
`String last = names.get(names.size() - 1);`	Gets the last element.

`ArrayList<Integer> squares = new ArrayList<Integer>();`
`for (int i = 0; i < 10; i++)`
`{`
`  squares.add(i * i);`
`}`
	Constructs an array list holding the first ten squares.

Wrappers

- You **cannot directly insert primitive type values** such as `int`, `char`, or `boolean` values–into array lists.
  - For example, you cannot form an `ArrayList<double>`.
  - You must use one of the **wrapper classes**

Primitive Type	Wrapper Class
byte	Byte
boolean	Boolean
char	Character
double	Double
float	Float
int	Integer
long	Long
short	Short

  - Note that the wrapper class names start with uppercase letters, and that two of them differ from the names of the corresponding primitive type: `Integer` and `Character`.
- To collect numbers in array lists, you must use wrapper classes.

Auto-boxing

- Conversion between primitive types and the corresponding wrapper classes is automatic.
- This process is called **auto-boxing**.
  - For example, if you assign a `double` value to a `Double` variable, the number is automatically **put into a box**.

```
Double wrapper = 29.95;
```

- Conversely, wrapper values are automatically **unboxed** to primitive types.

```
double x = wrapper;
```

Choosing Between Array Lists and Arrays

- Use an array
  - If the **size of a collection never changes**;
  - If you collect a **long sequence of primitive type values** and you are concerned about efficiency.
- Use an array list
  - Otherwise.
- Comparing

Operation	Arrays	Array Lists
Get an element.	`x = values[4];`	`x = values.get(4);`
Replace an element.	`values[4] = 35;`	`values.set(4, 35);`
Number of elements.	`values.length`	`values.size()`
Number of filled elements.	`currentSize` (companion variable, see Section 7.1.4)	`values.size()`
Remove an element.	See Section 7.3.6.	`values.remove(4);`
Add an element, growing the collection.	See Section 7.3.7.	`values.add(35);`
Initializing a collection.	`int[] values = { 1, 4, 9 };`	No initializer list syntax; call `add` three times.

Converting to ...

An array

- From a `String`

```
String testString = new String("This Is Test");
char[] stringToCharArray = testString.toCharArray();
```

- From an `ArrayList`

```
ArrayList<Integer> list = new ArrayList<Integer>();
...
int array[] = new int [list.size()];
for(int i =0; i < list.size();i++){
  array[i] = list.get(i);
}
```

```
ArrayList<Integer> list = new ArrayList<Integer>();
...
Integer[] resultArray = new Integer[list.size()];
resultArray = list.toArray(resultArray);
```

```
ArrayList<String> stringList = new ArrayList<String>();
...
String[] stringArray = new String[stringList.size()];
stringArray = stringList.toArray(stringList);
```

An Array List

- From an array of `String`

```
String[] array = {"Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep"
      , "Oct", "Nov", "Dec"};
ArrayList<String> list = new ArrayList<String>(Arrays.asList(array));
for(int i =0; i < list.size();i++){
  System.out.println(list.get(i));
}
```

- From an array of `int`

```
int arr[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
ArrayList<Integer> list = new ArrayList<Integer>();
for(int i =0; i < arr.length;i++){
  list.add(arr[i]);
}
for(int i =0; i < list.size();i++){
  System.out.println(list.get(i));
}
```

Random Numbers

Generate random numbers

- The Random class of the Java library implements a **random number generator** that produces numbers that appear to be completely random.
- To generate random numbers, you construct an object of the Random class, and then apply one of the following methods:
  - `nextInt(n)`: A random integer between the integers 0 (inclusive) and `n` (exclusive).
  - `nextDouble()`: A random floating-point number between 0 (inclusive) and 1 (exclusive).

```
Random randomGenerator = new Random();
for (int i = 0; i < 10; i++){
  int randomInt = 1 + randomGenerator.nextInt(100);
}
```

Check: `TP1_12`

Programming Issues

Overflow

- A numeric computation overflows if the result falls outside the range for the number type.

```
int n = 1000000;
System.out.println(n * n); // Prints -727379968, which is clearly wrong
```

  - The product $n * n$ is 10^{12}, which is larger than the largest integer (about $2 * 10^9$).
  - The result is truncated to fit into an `int`.
- There is no warning when an integer overflow occurs.

Floating-point representation

- Rounding errors occur when an exact representation of a floating-point number is not possible.

```
double f = 4.35;
System.out.println(100 * f); // Prints 434.99999999999994
```

- The problem arises because computers represent numbers in the binary number system.
  - In the binary number system, there is no exact representation of the fraction $1/10$, just as there is no exact representation of the fraction $1/3 = 0.33333$ in the decimal number system.
- For this reason, the `double` type is not appropriate for financial calculations.

Unintended Integer Division

- / is used for both integer and floating-point division.
- It is a common error to use integer division by accident.

```
int score1 = 10;
int score2 = 4;
int score3 = 9;
double average = (score1 + score2 + score3) / 3; // Error
System.out.println("Average score: " + average); // Prints 7.0, not
     7.666666666666667
```

- Solution:

```
double total = score1 + score2 + score3;
double average = total / 3;
```

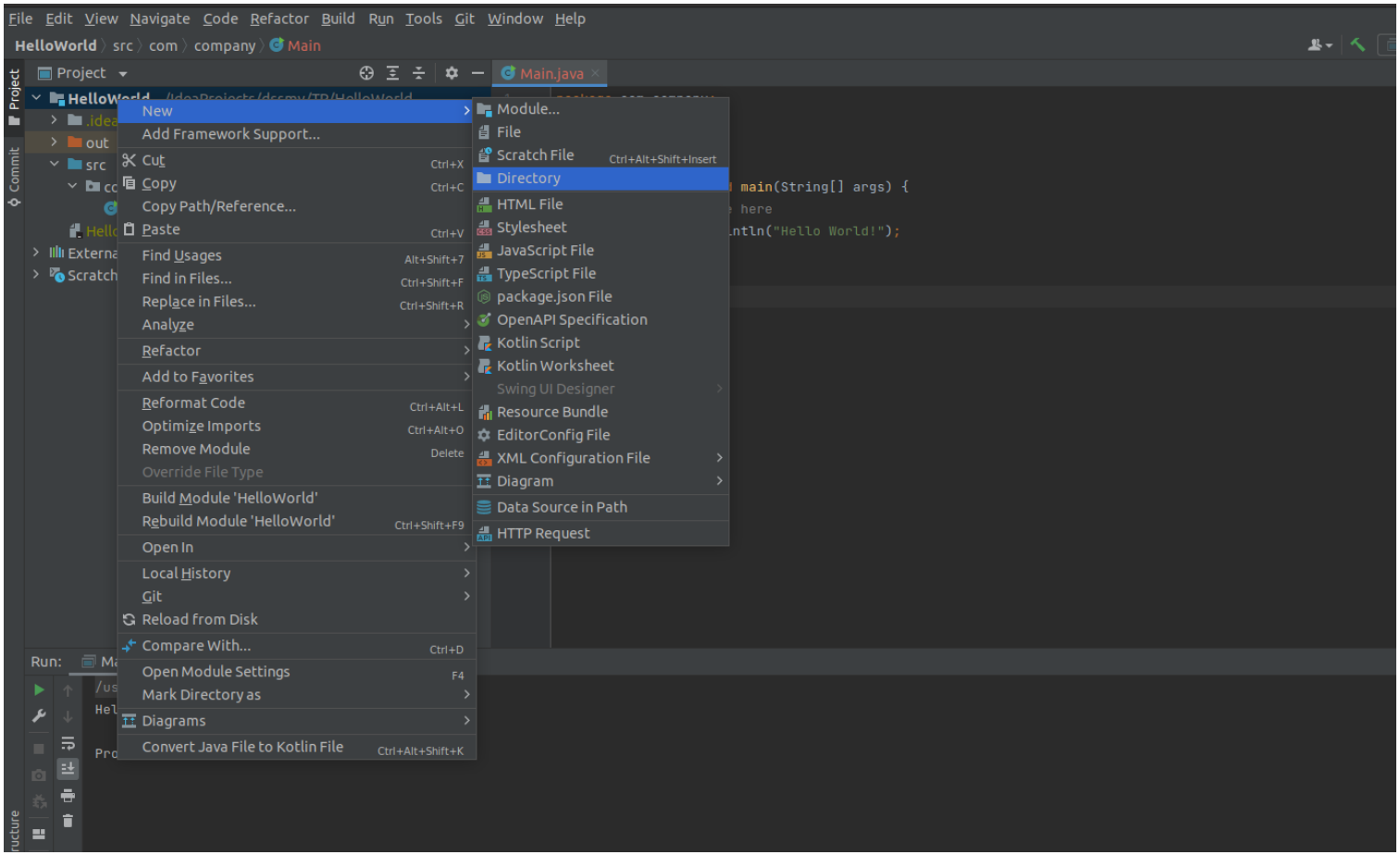
or

```
double average = (score1 + score2 + score3) / 3.0;
```

Unit Tests

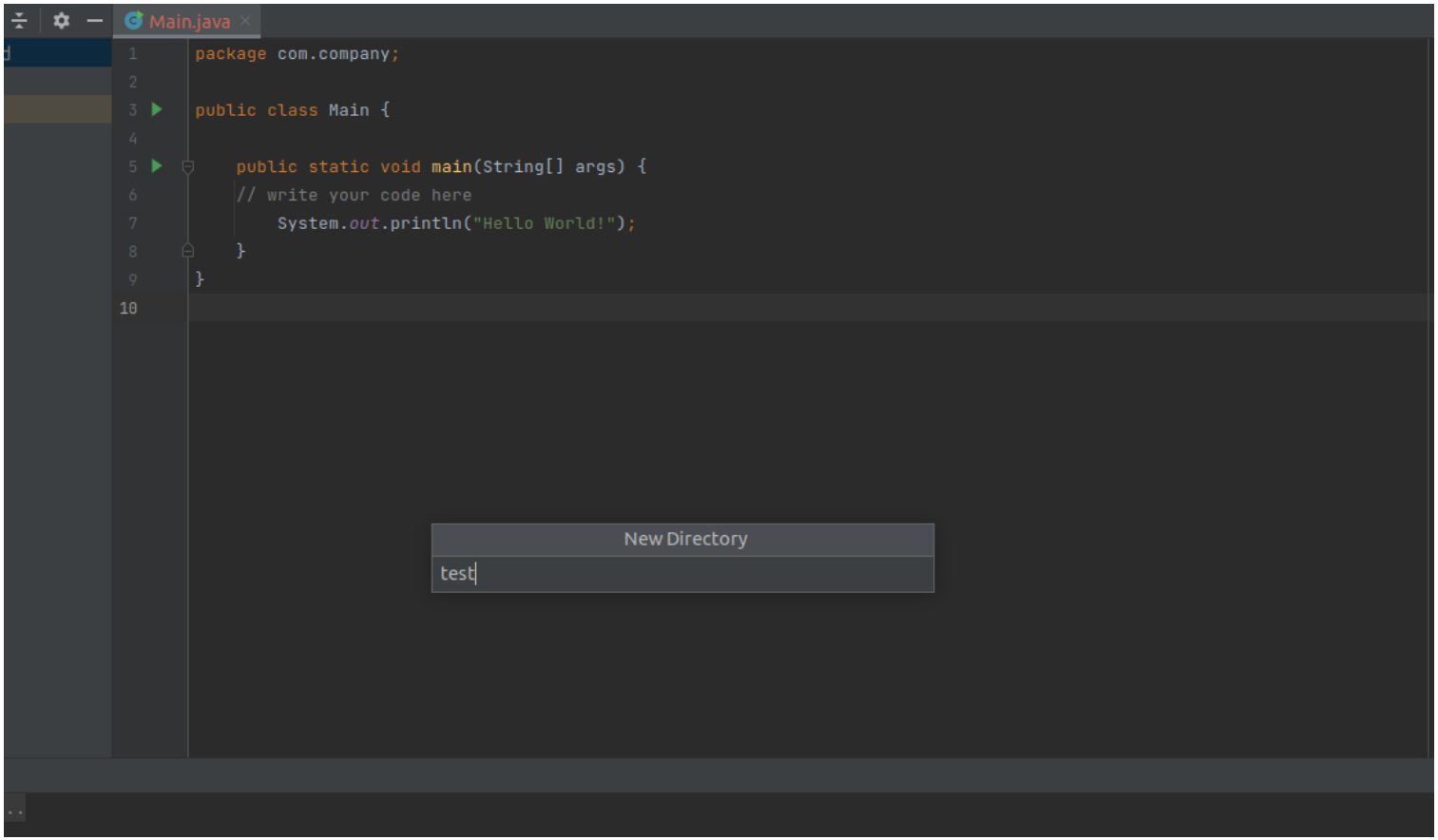
Create a `test` folder (I)

- Right click on your `HelloWorld` project and select `New > Directory`.



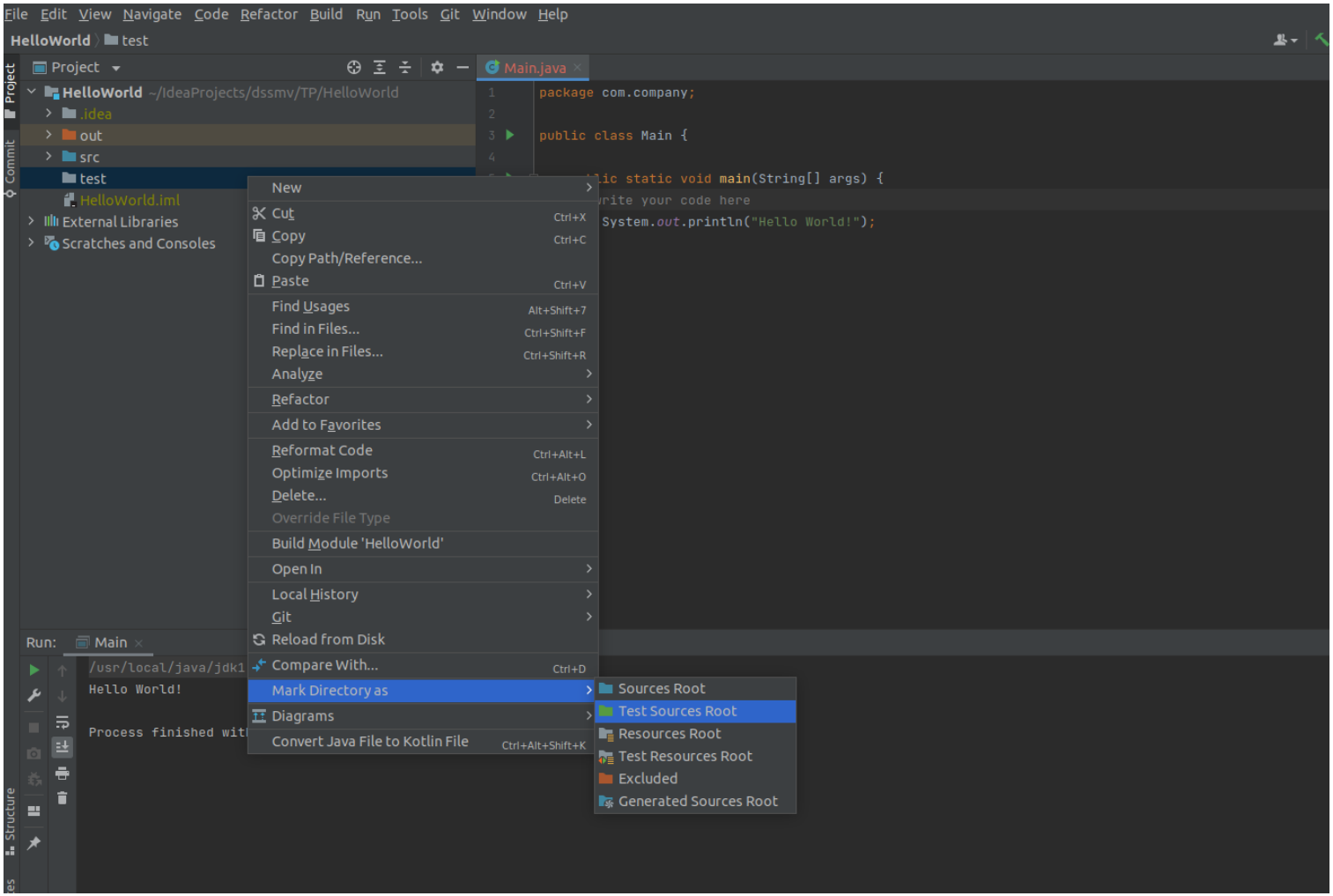
Create a `test` folder (II)

- Enter `test` name.



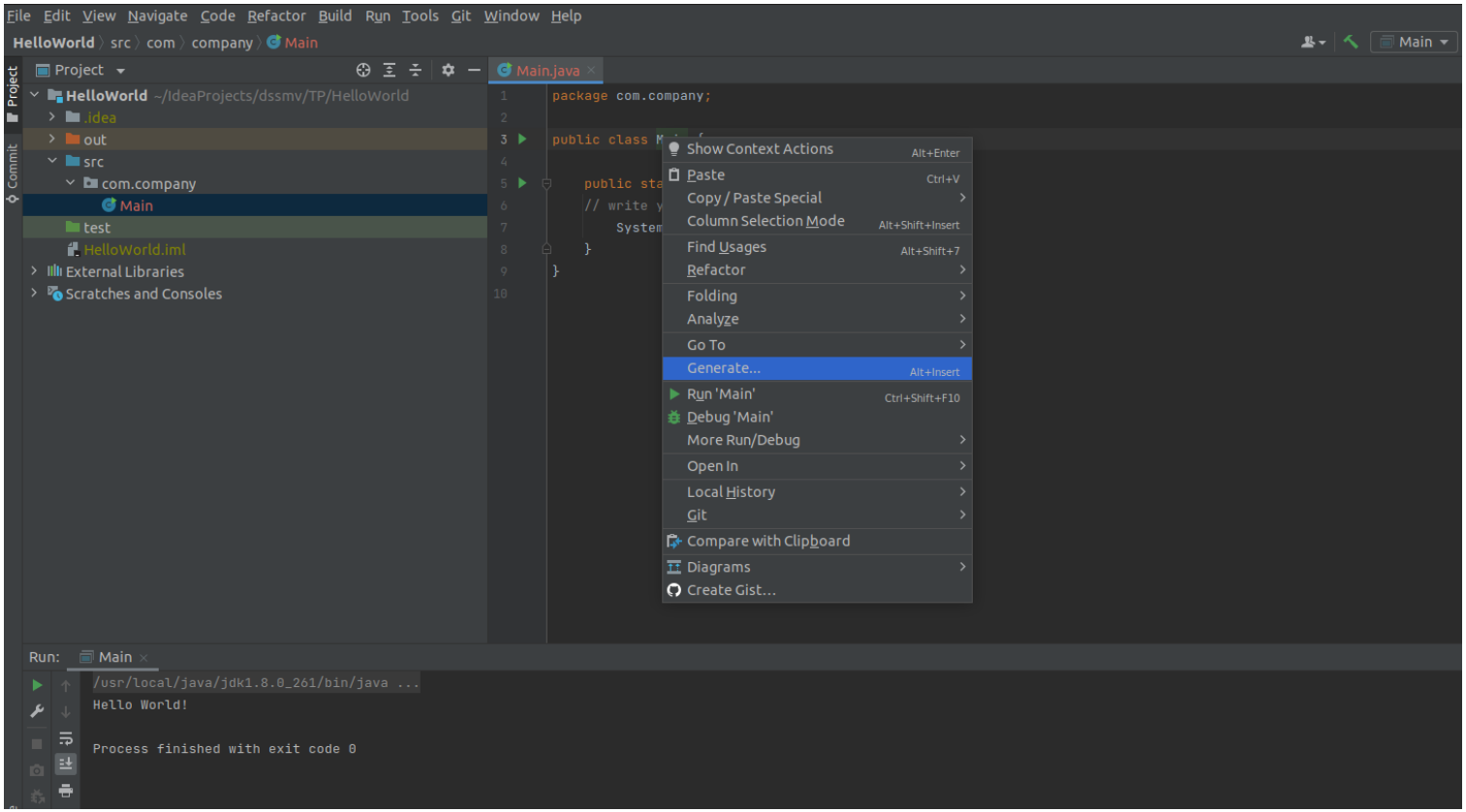
Configure `test` folder as test root

- Right click on `test` folder and select `Mark Directory as > Test Sources Root`.



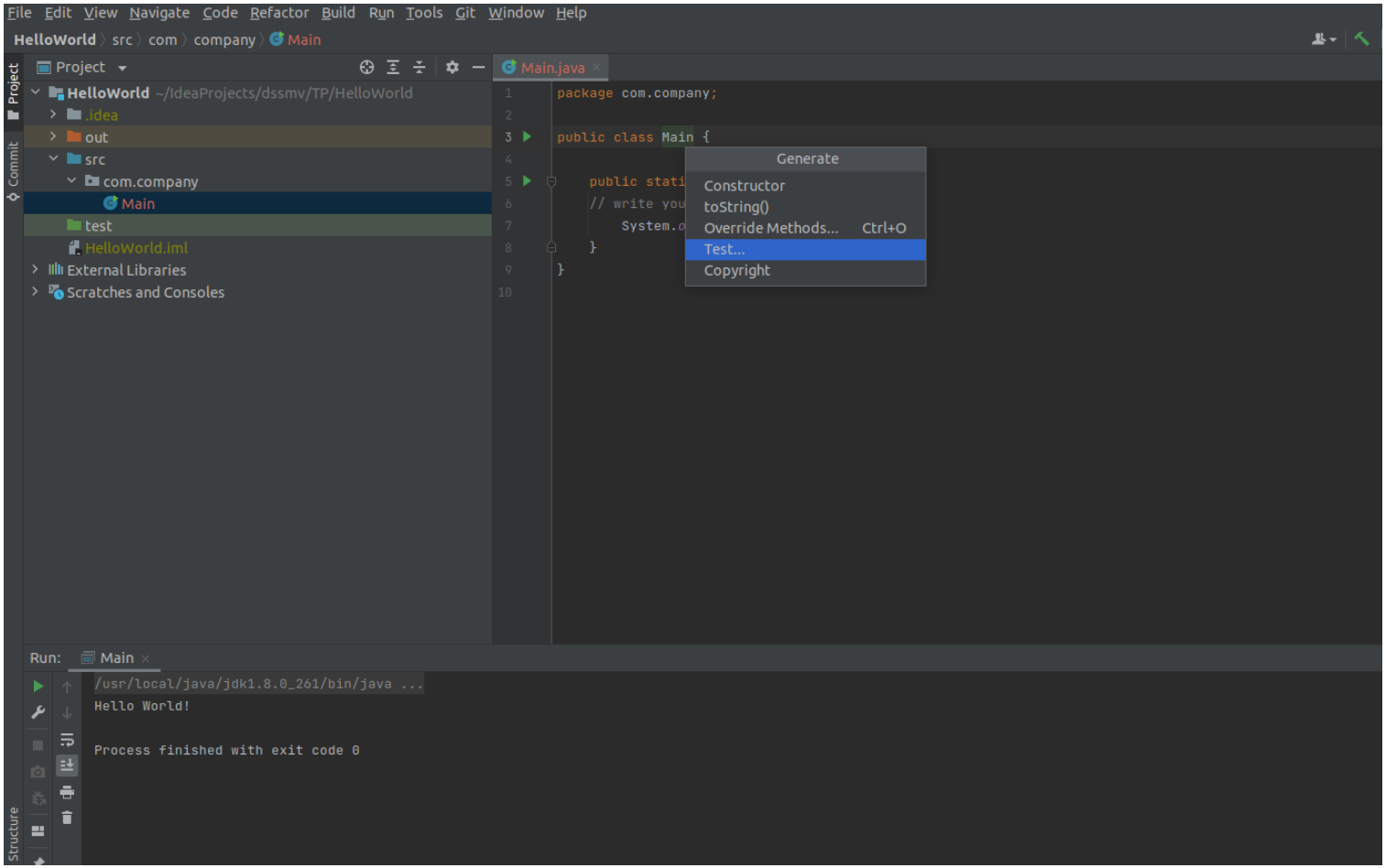
Create the `MainTest` class (I)

- Right click on `Main` class and select `Generate`.



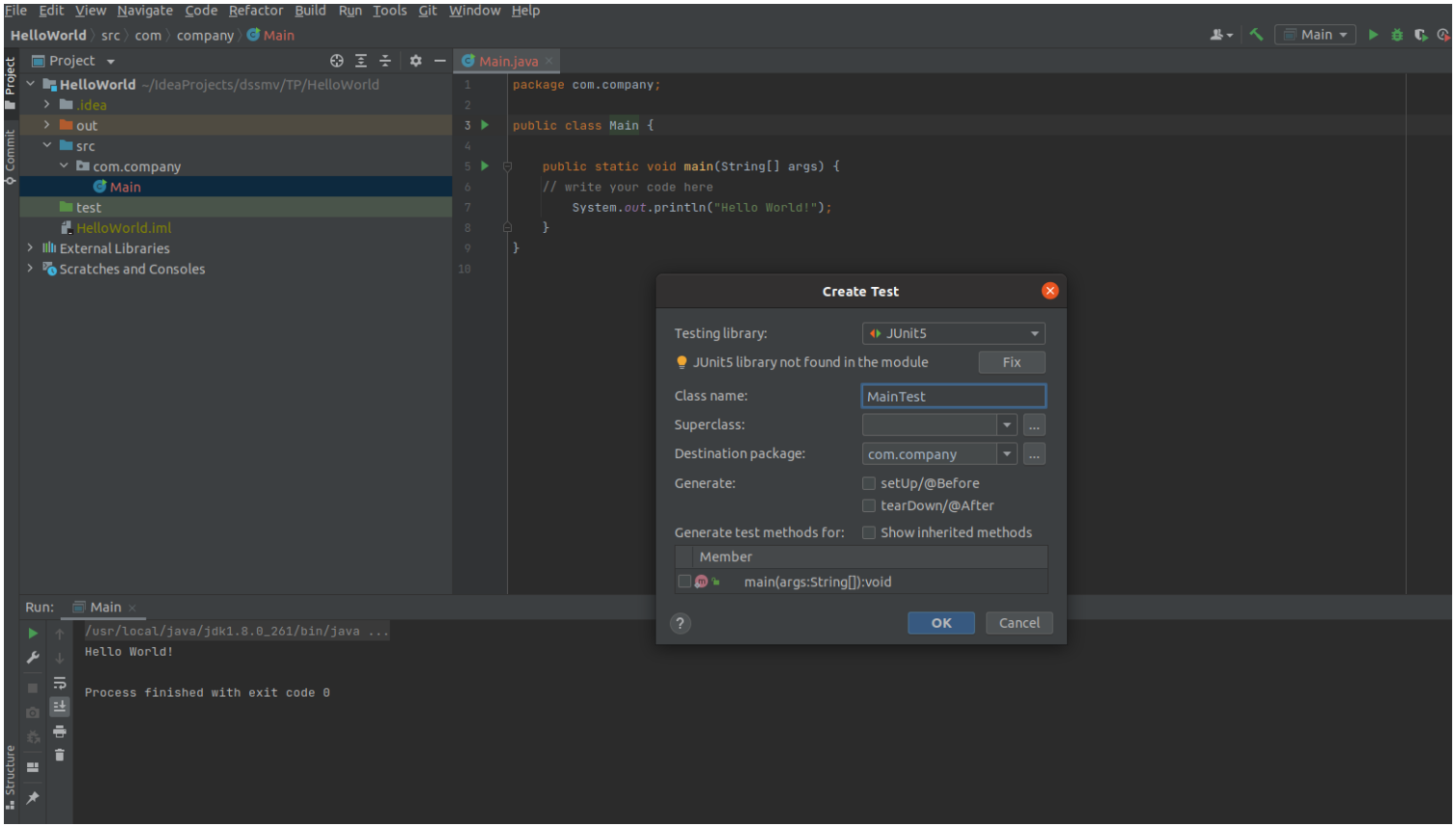
Create the `MainTest` class (II)

- Select `Test...`

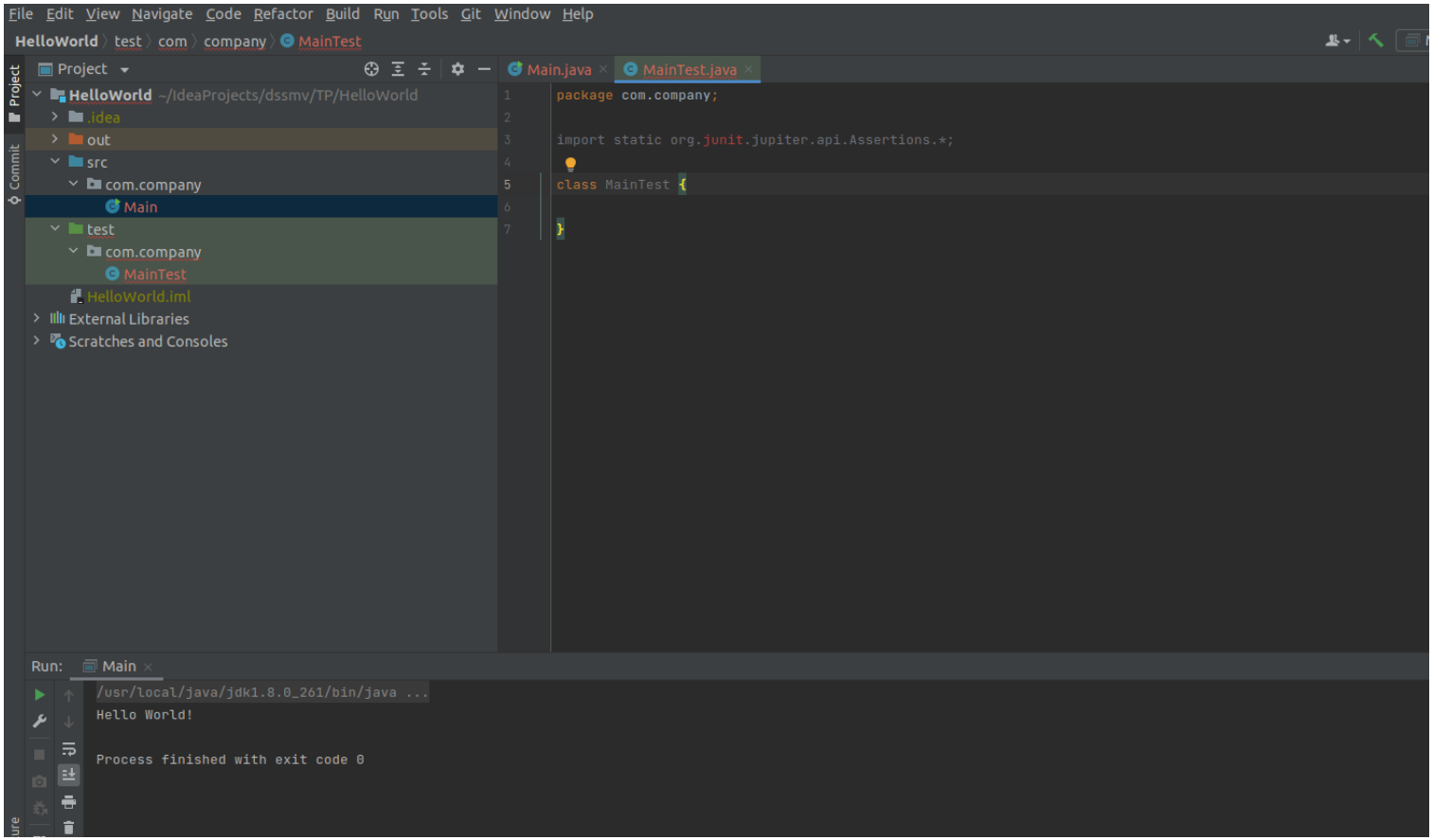


Create the `MainTest` class (III)

- Configure `MainTest` class.



Create the `MainTest` class (IV)



Adding code (I)

- Add `isEven` method to class `Main`

```
package com.company;

public class Main {

  public static boolean isEven(int n){
    boolean res = n % 2 == 0 ? true : false;
    return res;
  }

  public static void main(String[] args) {
    // write your code here
    System.out.println("Hello World!");
  }
  }
```

Adding code (II)

- Add `isEven` method test to class `MainTest`

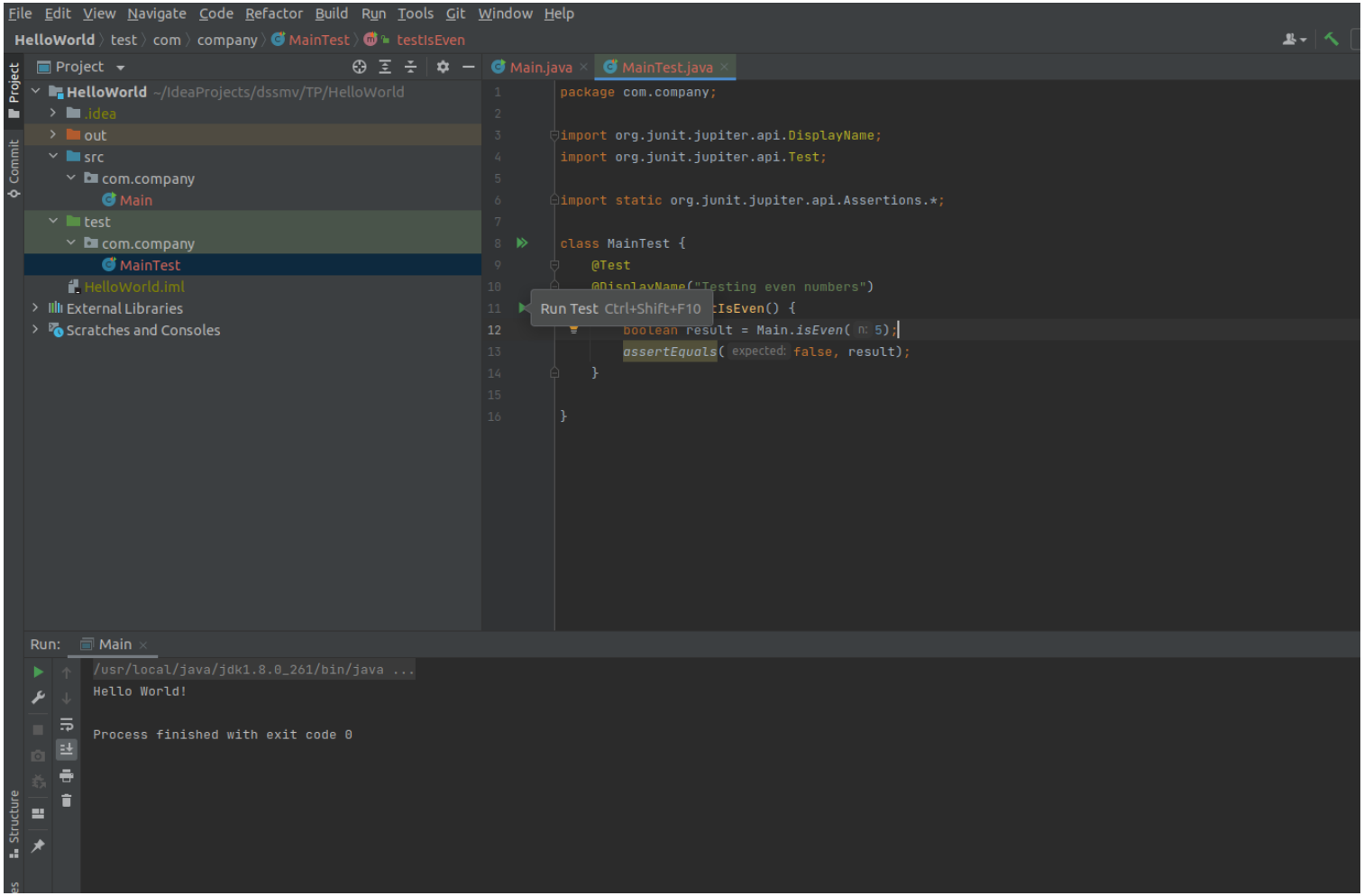
```
package com.company;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class MainTest {
  @Test
  @DisplayName("Testing even numbers")
  public void testIsEven() {
    boolean result = Main.isEven(5);
    assertEquals(false, result);
  }
}
```

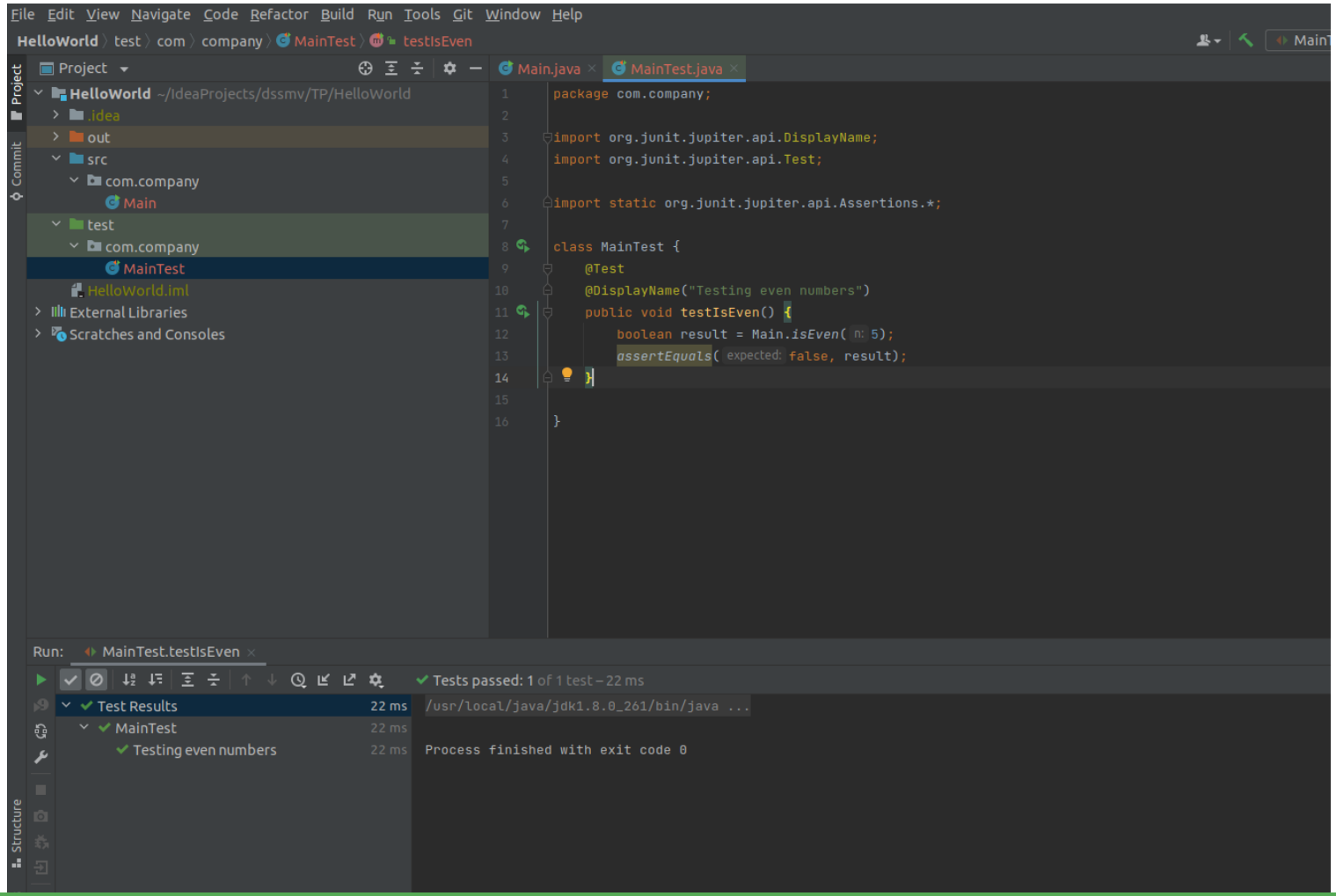
Running tests (I)

- Left click on green arrow.



Running tests (II)

- JUnit doc available at: `https://junit.org/junit5/docs/current/user-guide/`



Using a Debugger

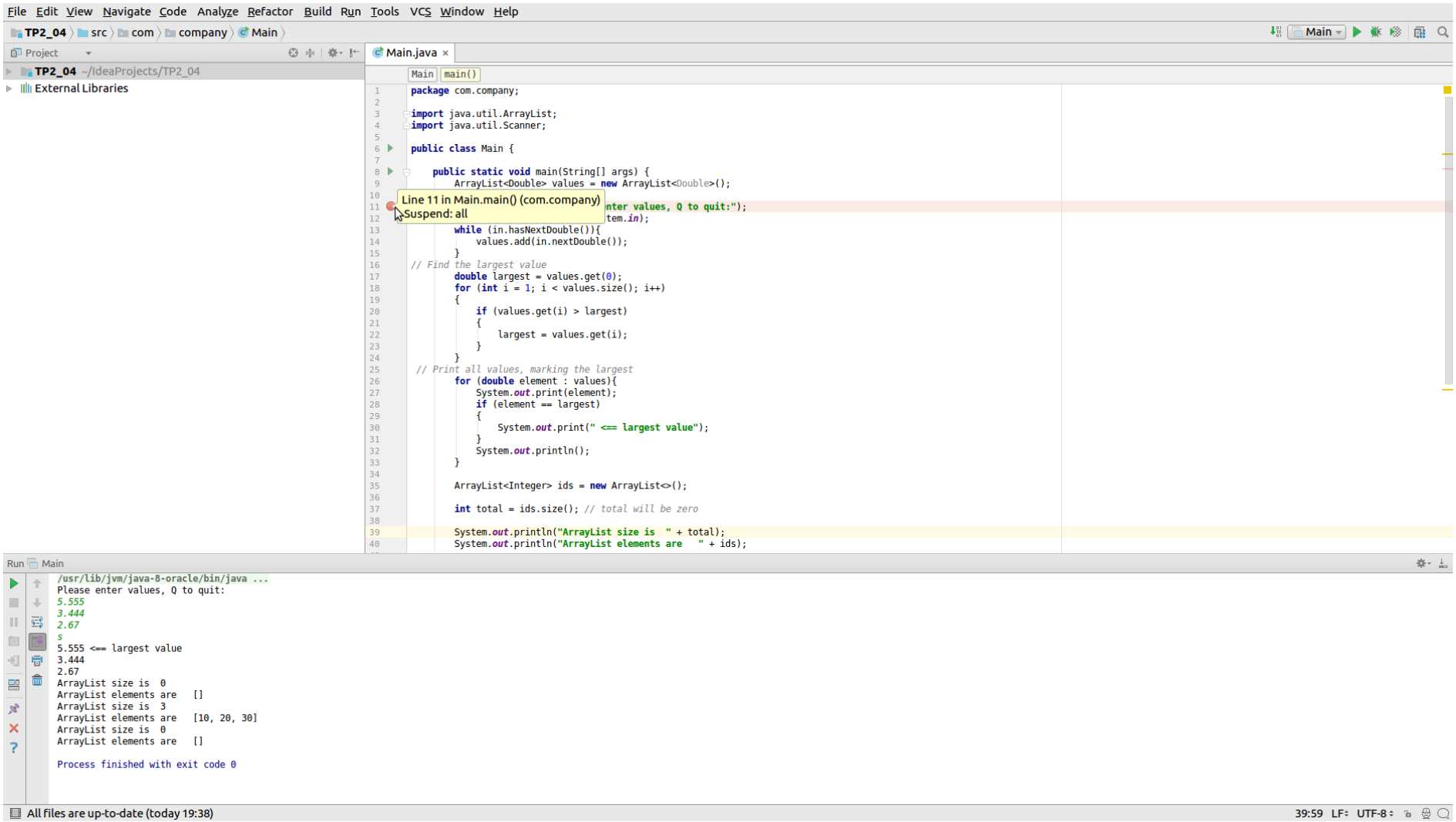
Debugging

- As you have undoubtedly realized by now, computer programs rarely run perfectly the first time.
- At times, it can be quite frustrating to find the bugs. Of course, you can insert print commands, run the program, and try to analyze the printout. If the printout does not clearly point to the problem, you may need to add and remove print commands and run the program again.
- Modern development environments contain special programs, called **debuggers**, that help you locate bugs by letting you follow the execution of a program.
  - You can stop and restart your program and see the contents of variables whenever your program is temporarily stopped.
  - At each stop, you have the choice of what variables to inspect and how many program steps to run until the next stop.

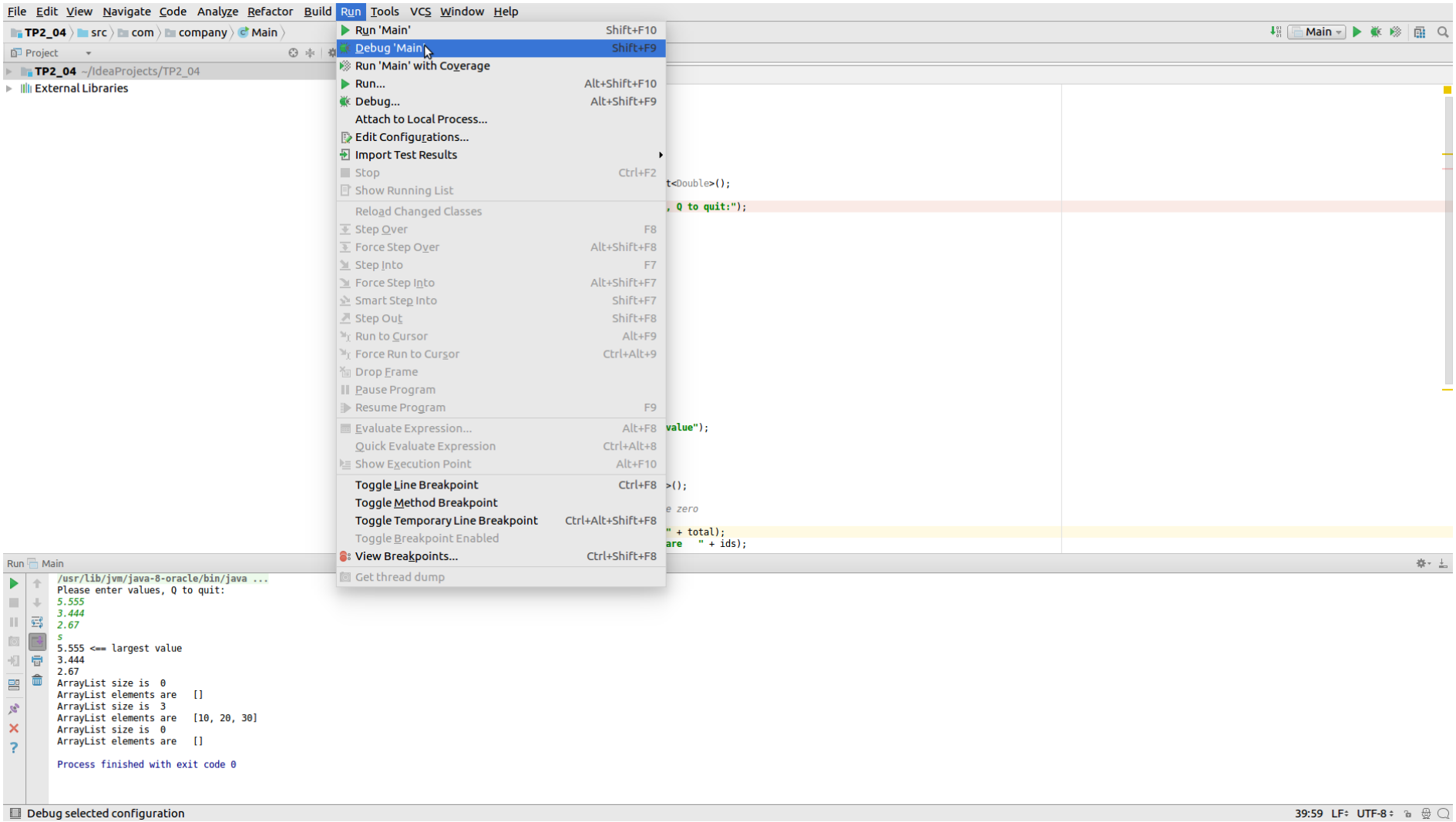
Check: `https://www.jetbrains.com/help/idea/2016.3/debugging.html`

Check: `https://www.youtube.com/watch?v=VdBsUv4lnm4`

Debugging with IntelliJ IDEA (I)



Debugging with IntelliJ IDEA (II)



Debugging with IntelliJ IDEA (III)

